

AWS en pratique

Conteneurs : ECR, ECS sur Fargate, découverte de EKS

Images et registre. ECS sur Fargate. Exploiter un service ECS. Découverte de EKS.

Images Docker, Dockerfile	ECR dépôts, scan, architectures	ECS tâche, service, Fargate	Rôles exécution, tâche, journaux	Exploitation ALB, déploiement, scaling	EKS Kubernetes managé
-------------------------------------	---	---------------------------------------	--	--	---------------------------------

Support de formation, version 1.2, octobre 2026. Parcours « AWS en pratique ». Fait suite au support « Bases de données ».

Images et registre

- Rappeler ce que sont une image, un conteneur et un Dockerfile.
- Pousser une image dans ECR ; activer le scan et une règle de cycle de vie.
- Éviter l'erreur d'architecture entre x86_64 et ARM64.

Exploiter un service ECS

- Exposer un service par un Application Load Balancer.
- Déployer par mise à jour progressive, avec disjoncteur et retour arrière.
- Faire varier le nombre de tâches automatiquement.

Prérequis. Les supports précédents : rôles IAM et `iam:PassRole` , VPC et security groups, load balancer, Auto Scaling, Secrets Manager. Avoir déjà lancé un conteneur avec Docker aide beaucoup.

ECS sur Fargate

- Distinguer cluster, task definition, tâche et service.
- Choisir entre Fargate et EC2.
- Distinguer rôle d'exécution et rôle de tâche ; envoyer les journaux vers CloudWatch.

Découverte de EKS

- Décrire le plan de contrôle et les nœuds d'un cluster Kubernetes managé.
- Expliquer l'accès au cluster et les rôles IAM pour les pods.
- Choisir entre ECS et EKS d'après des critères.

Fil rouge. L'API de UtopiosShop, jusqu'ici sur des instances, est empaquetée dans une image et tourne en conteneurs sur ECS et Fargate.

Images et registre

Rappels Docker	Image, conteneur, Dockerfile.
Amazon ECR	Dépôts, authentification, scan, cycle de vie.
Architectures	x86_64, ARM64, images multi-architecture.

Exploiter un service ECS

Exposition	Service derrière un ALB.
Déploiement	Mise à jour progressive, disjoncteur.
Auto scaling	Nombre de tâches selon la charge.

ECS sur Fargate

Amazon ECS	Cluster, task definition, tâche, service.
Fargate ou EC2	Qui gère les machines.
Rôles et journaux	Exécution, tâche, CloudWatch Logs.

Découverte de EKS

Amazon EKS	Plan de contrôle, nœuds.
Accès et rôles	Accès au cluster, rôles des pods.
ECS ou EKS	Critères de choix.

Convention. Les tarifs cités sont ceux de la région us-east-1, hors taxes, à titre d'ordre de grandeur. La page de tarification de chaque service fait foi.

Images et registre, partie 1

Rappels Docker

Empaqueter une application avec tout ce qu'il lui faut pour tourner, de façon identique sur un poste, en test et en production.

- Pourquoi les conteneurs
- Conteneur et machine virtuelle
- L'image et ses couches
- Le Dockerfile
- Construire, nommer, pousser
- Bonnes pratiques

Définition : conteneur

Processus isolé du reste du système, qui s'exécute avec son propre système de fichiers, ses bibliothèques et sa configuration, tout en partageant le noyau de la machine hôte.

Définition : image

Modèle en lecture seule à partir duquel on lance des conteneurs : l'application, ses dépendances et le strict nécessaire du système, figés dans un paquet versionné.

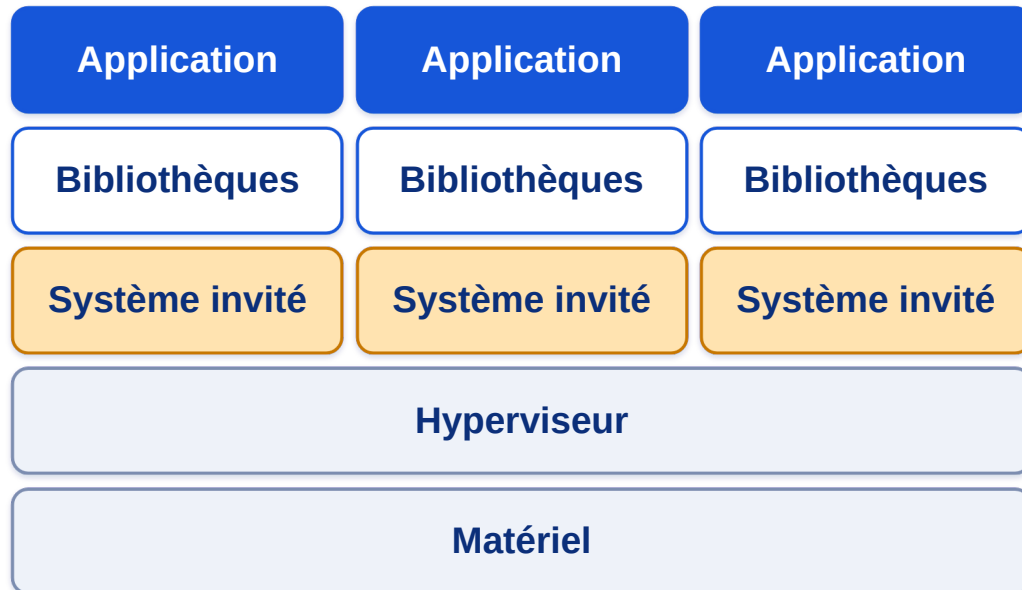
Image et conteneur. L'image est au conteneur ce que l'AMI est à l'instance : le modèle, et son exemplaire en cours d'exécution.

Problème	Ce que le conteneur apporte
« Ça marche sur mon poste »	La même image tourne partout, à l'identique.
Conflits de versions entre applications	Chaque conteneur embarque ses propres dépendances.
Serveurs longs à préparer	Un conteneur démarre en quelques secondes.
Machines sous-utilisées	Plusieurs conteneurs partagent une machine.
Déploiements risqués	Revenir en arrière, c'est relancer l'image précédente.

Lien avec le chapitre « Élasticité ». Un conteneur est jetable par nature : on ne le répare pas, on le remplace. C'est le principe « sans état », poussé jusqu'au bout.

Schéma : machine virtuelle et conteneur

Machines virtuelles



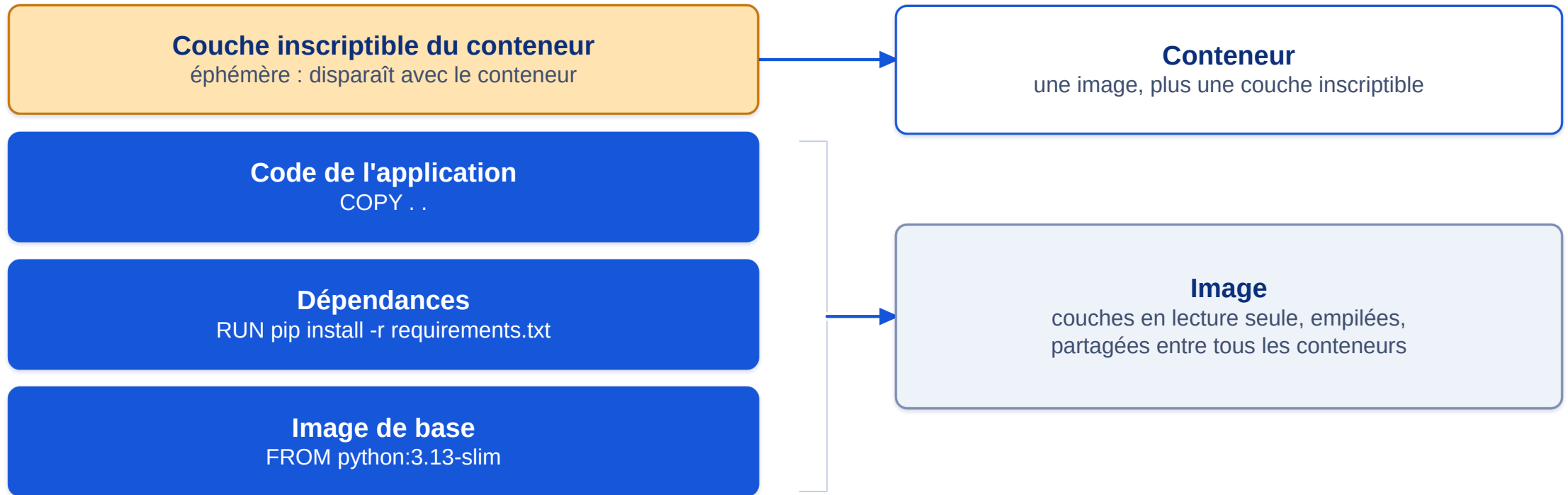
Conteneurs



Lecture. Une machine virtuelle embarque un système d'exploitation complet : plusieurs gigaoctets, un démarrage en minutes. Un conteneur n'embarque que l'application et ses bibliothèques : quelques dizaines de mégaoctets, un démarrage en secondes.

Les deux se combinent. Sur AWS, les conteneurs tournent dans des machines virtuelles : des instances EC2, ou les micro-machines de Fargate. L'isolation forte vient de la machine virtuelle, la légèreté du conteneur.

Schéma : l'image et ses couches

**Définition : couche (layer)**

Ensemble de fichiers ajoutés ou modifiés par une instruction du Dockerfile. Une couche déjà présente n'est ni reconstruite, ni retransférée.

Deux conséquences. L'ordre des instructions compte : ce qui change rarement va en bas, pour profiter du cache. Et tout ce qu'un conteneur écrit sur son disque disparaît avec lui : les données vont dans une base, S3 ou EFS.

`dockerfile` Dockerfile de l'API de UtopiosShop (Python)

```
# 1. Image de base : petite, et de version précise
FROM python:3.13-slim

# 2. Dossier de travail dans l'image
WORKDIR /app

# 3. Dépendances d'abord : cette couche ne change
#     que si requirements.txt change
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 4. Puis le code, qui change souvent
COPY . .

# 5. Ne pas tourner en root
RUN useradd --create-home appuser
USER appuser

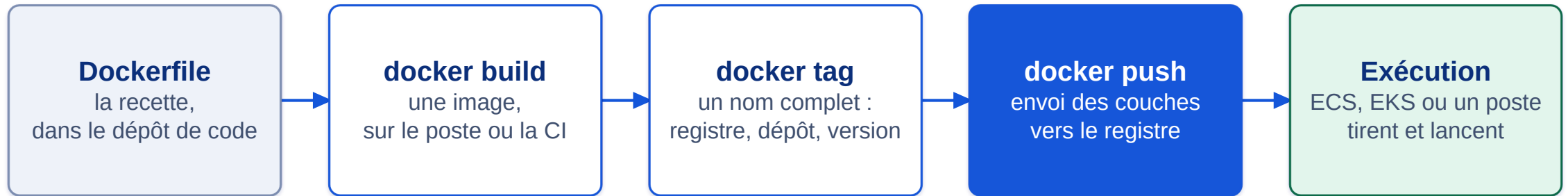
# 6. Port d'écoute (documentation) et commande
EXPOSE 8080
CMD ["gunicorn", "--bind", "0.0.0.0:8080", "app:app"]
```

Définition : Dockerfile

Fichier texte qui décrit, instruction par instruction, comment construire une image. Il se versionne avec le code de l'application.

Instruction	Rôle
FROM	L'image de départ.
WORKDIR	Le dossier courant.
COPY	Copier des fichiers dans l'image.
RUN	Exécuter une commande à la construction.
ENV	Définir une variable d'environnement.
USER	L'utilisateur qui exécutera l'application.
EXPOSE	Documenter le port d'écoute.
CMD, ENTRYPOINT	La commande lancée au démarrage du conteneur.

Schéma : de la recette à l'exécution



L'image construite une fois est celle qui tourne partout : test, préproduction, production.

On ne reconstruit pas pour déployer ; on déploie ce qui a été testé.

Définition : registre (registry)

Service qui stocke et distribue des images de conteneurs. Docker Hub est le registre public le plus connu ; Amazon ECR est le registre d'AWS.

Le registre est le point de passage obligé. Une machine qui doit lancer un conteneur tire l'image depuis le registre : sans lui, l'image reste sur le poste qui l'a construite.

Schéma : le nom complet d'une image

123456789012.dkr.ecr.eu-west-3.amazonaws.com/shop/api:1.4.2

registre : compte et région

dépôt

tag

Sans registre indiqué, Docker cherche sur Docker Hub. Sans tag, il suppose « latest ».

Forme exacte, par empreinte : shop/api@sha256:9f2c... désigne un contenu précis, qui ne change jamais.

Définition : tag

Étiquette lisible attachée à une version d'image dans un dépôt : `1.4.2`, `prod`, `latest`. Un tag peut être déplacé vers une autre version, sauf si le dépôt l'interdit.

Définition : empreinte (digest)

Identifiant calculé à partir du contenu de l'image, de la forme `sha256:...`. Deux images de même empreinte sont strictement identiques.

\$ Prérequis : Docker installé et démarré ; un dossier contenant le Dockerfile et le code

Construire l'image depuis le dossier courant, et la nommer

```
docker build -t shop-api:1.4.2 .
```

Lister les images locales, avec leur taille

```
docker images
```

Lancer un conteneur : port 8080 de la machine relié au port 8080 du conteneur, une variable d'environnement

```
docker run --rm -d --name api -p 8080:8080 -e LOG_LEVEL=info shop-api:1.4.2
```

Voir les conteneurs en cours, leurs journaux, et y ouvrir un terminal

```
docker ps
```

```
docker logs -f api
```

```
docker exec -it api /bin/sh
```

Arrêter le conteneur

```
docker stop api
```

Inspecter une image : architecture, utilisateur, commande de démarrage

```
docker image inspect shop-api:1.4.2 --format '{{.Architecture}} {{.Config.User}} {{.Config.Cmd}}'
```

Voir les couches d'une image et leur taille

```
docker history shop-api:1.4.2
```

Tester en local avant de pousser. Si l'image ne démarre pas avec `docker run`, elle ne démarrera pas mieux sur ECS : c'est le premier diagnostic, et le plus rapide.

Bonnes pratiques

- **Image de base réduite** : variantes `slim`, `alpine` ou « distroless ». Moins de paquets, moins de failles.
- **Version précise** de l'image de base : `python:3.13-slim`, pas `python:latest`.
- **Construction en plusieurs étapes** : compiler dans une image, ne garder que le résultat dans l'image finale.
- **Utilisateur non privilégié** : instruction `USER`.
- **Fichier `.dockerignore`** : ne pas embarquer `.git`, dépendances locales, secrets.
- **Un processus par conteneur**, qui écrit ses journaux sur la sortie standard.
- **Configuration par variables d'environnement** : la même image pour tous les environnements.

À ne pas faire	Pourquoi
Secret dans le Dockerfile ou l'image	Lisible par quiconque tire l'image, à jamais dans ses couches.
Tag <code>latest</code> en production	On ne sait plus quelle version tourne ; retour arrière impossible.
Écrire des données dans le conteneur	Perdues à son arrêt.
Tourner en root	Une faille de l'application donne les pleins pouvoirs dans le conteneur.
Installer des outils de débogage	Surface d'attaque et taille inutiles.
Une image par environnement	Ce qui est testé n'est pas ce qui est déployé.

Erreur	Symptôme	Correction
Application qui écoute sur 127.0.0.1	Injoignable de l'extérieur du conteneur.	Écouter sur <code>0.0.0.0</code> .
<code>COPY . .</code> avant l'installation des dépendances	Dépendances réinstallées à chaque modification du code.	Copier d'abord le fichier de dépendances.
Pas de <code>.dockerignore</code>	Image énorme, secrets embarqués.	Exclure ce qui n'a rien à faire dans l'image.
Processus lancé en arrière-plan dans CMD	Le conteneur s'arrête aussitôt.	Le processus principal reste au premier plan.
Journaux écrits dans un fichier	Invisibles pour la plateforme.	Sortie standard.
Signal d'arrêt ignoré	Arrêt brutal après le délai, requêtes coupées.	Forme « exec » de CMD ; traiter SIGTERM.
Image construite sur un poste, jamais dans une chaîne	Résultat dépendant de la machine de chacun.	Construction automatisée, reproductible.

Images et registre, partie 2

Amazon ECR

Le registre d'images d'AWS : privé, intégré à IAM, et placé au plus près des services qui lancent les conteneurs.

- Registre et dépôt
- L'authentification
- Pousser et tirer
- Le scan des images
- Tags immuables et cycle de vie
- Accès, coût, erreurs fréquentes

Définition : Amazon ECR (Elastic Container Registry)

Registre d'images de conteneurs managé. Chaque compte dispose, dans chaque région, d'un registre privé où il crée ses dépôts.

Définition : dépôt (repository)

Emplacement d'un registre qui contient toutes les versions d'une même image, distinguées par leurs tags et leurs empreintes. Un dépôt par application ou par composant.

Propriété	Détail
Privé par défaut	Accès contrôlé par IAM et par une politique de dépôt.
Régional	Les images sont dans la région du dépôt ; réplication possible vers d'autres.
Chiffré	Au repos, d'office ; par une clé KMS si on le souhaite.
Intégré	ECS, EKS et Lambda y tirent leurs images grâce à leurs rôles.
Standard	Compatible avec Docker et tous les outils du format OCI.

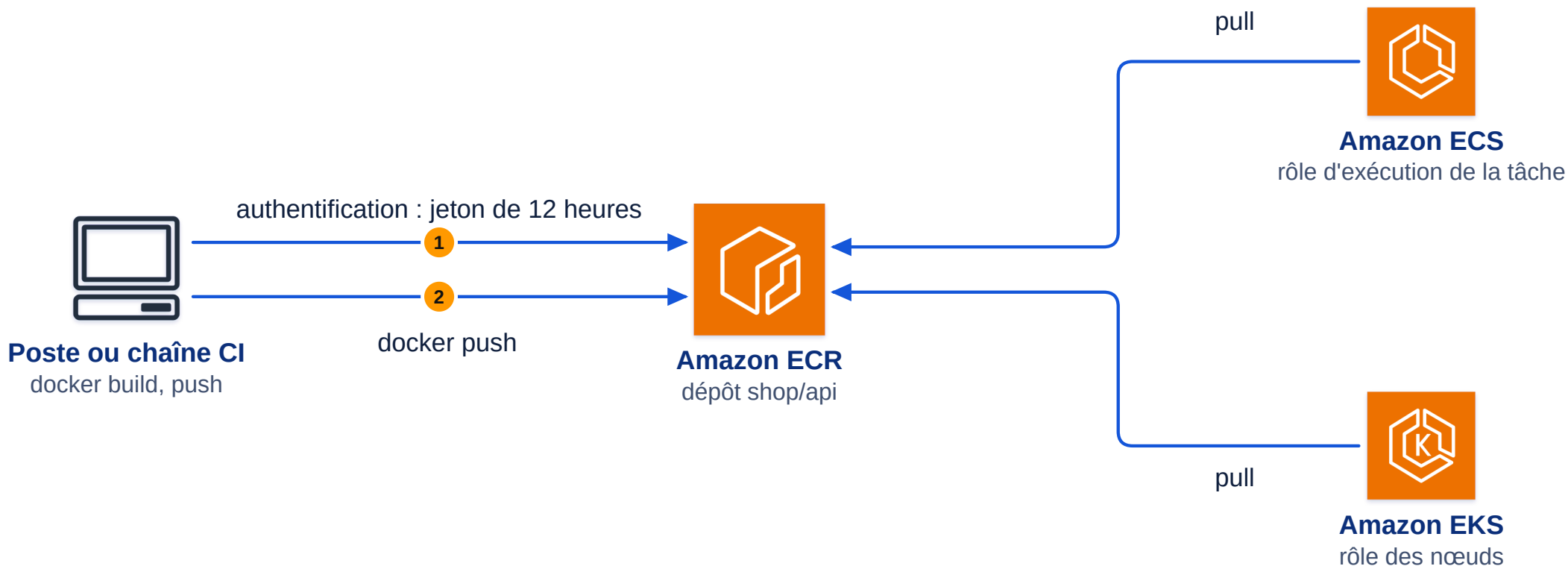
Adresse
Registre : <code><compte>.dkr.ecr.<région>.amazonaws.com</code>
Image : <code>...amazonaws.com/shop/api:1.4.2</code>

Pourquoi pas Docker Hub

- Les images restent dans votre compte, sous vos droits IAM.
- Aucun identifiant externe à gérer ni à faire tourner.
- Pas de limite de débit sur les téléchargements.
- Transfert rapide et gratuit vers les services de la même région.

ECR Public est une galerie distincte, pour diffuser des images à tous. On y trouve les images officielles d'AWS.

Schéma : pousser et tirer



1. Le poste, ou la chaîne d'intégration, échange ses identifiants AWS contre un jeton que Docker utilise comme mot de passe, valable 12 heures.

2. Docker pousse les couches de l'image dans le dépôt ; seules celles qui n'y sont pas encore sont envoyées.

L'authentification auprès d'ECR

\$ Prérequis : AWS CLI connectée, Docker démarré

```
ACCOUNT_ID=$(aws sts get-caller-identity \
  --query Account --output text)
REGISTRE=$ACCOUNT_ID.dkr.ecr.eu-west-3.amazonaws.com
```

```
# Obtenir un jeton et le passer à Docker
aws ecr get-login-password --region eu-west-3 \
  | docker login --username AWS \
    --password-stdin $REGISTRE
# Login Succeeded
```

Sous Windows : « error storing credentials ». Dans le fichier `%USERPROFILE%\docker\config.json`, retirer la ligne `"credsStore"`, laisser une entrée `"auths": {}`, puis relancer la commande de connexion.

Point	Détail
Utilisateur	Toujours <code>AWS</code> .
Mot de passe	Un jeton temporaire, valable 12 heures.
Droit requis	<code>ecr:GetAuthorizationToken</code> , puis les droits de lecture ou d'écriture sur le dépôt.
Portée	Un registre : un compte et une région.

Les services ne font pas de « docker login ». ECS et EKS s'authentifient d'eux-mêmes avec leur rôle : il suffit que ce rôle ait le droit de tirer l'image.

Sous PowerShell, la commande est la même ; les variables s'écrivent `$ACCOUNT_ID = aws sts get-caller-identity --query Account --output text` et `$REGISTRE = "$ACCOUNT_ID.dkr.ecr.eu-west-3.amazonaws.com"`.

Ligne de commande : créer un dépôt et pousser une image

\$ Prérequis : connexion Docker à ECR établie (diapositive précédente), REGISTRE défini ; remplacer user7

1. Créer le dépôt : scan à chaque envoi, tags immuables

```
aws ecr create-repository --repository-name user7/shop-api \  
  --image-scanning-configuration scanOnPush=true --image-tag-mutability IMMUTABLE
```

2. Construire l'image pour l'architecture visée (ici x86_64), puis lui donner son nom complet

```
docker build --platform linux/amd64 -t shop-api:1.4.2 .  
docker tag shop-api:1.4.2 $REGISTRE/user7/shop-api:1.4.2
```

3. Pousser

```
docker push $REGISTRE/user7/shop-api:1.4.2
```

4. Lister les images du dépôt : tags, empreinte, taille, date

```
aws ecr describe-images --repository-name user7/shop-api --output table \  
  --query 'reverse(sort_by(imageDetails,&imagePushedAt))[][imageTags[0],imageDigest,imageSizeInBytes,imagePushedAt]'
```

5. Tirer l'image depuis une autre machine

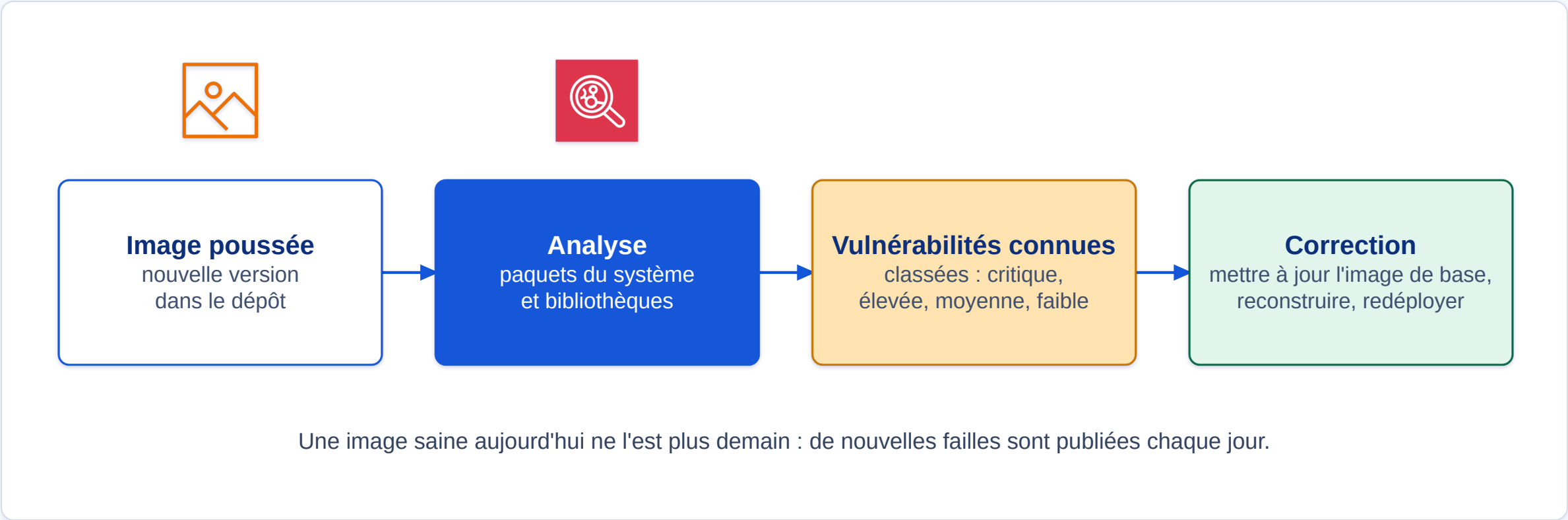
```
docker pull $REGISTRE/user7/shop-api:1.4.2
```

6. Supprimer une image, puis le dépôt et tout son contenu

```
aws ecr batch-delete-image --repository-name user7/shop-api --image-ids imageTag=1.4.2  
aws ecr delete-repository --repository-name user7/shop-api --force
```

Le dépôt doit exister avant le premier envoi : ECR ne le crée pas à la volée, sauf si un modèle de création de dépôt a été configuré pour le registre.

Schéma : le scan des images



\$ Prérequis : image poussée dans un dépôt où le scan est activé

```
# Attendre la fin de l'analyse
aws ecr wait image-scan-complete \
  --repository-name user7/shop-api --image-id imageTag=1.4.2

# Nombre de vulnérabilités par gravité
aws ecr describe-image-scan-findings \
  --repository-name user7/shop-api --image-id imageTag=1.4.2 \
  --query 'imageScanFindings.findingSeverityCounts'

# Détail des plus graves : identifiant, paquet, version
aws ecr describe-image-scan-findings \
  --repository-name user7/shop-api --image-id imageTag=1.4.2 \
  --output table --query \
  'imageScanFindings.findings[?severity==`CRITICAL`].[name,
  attributes[?key==`package_name`][0].value,
  attributes[?key==`package_version`][0].value]'
```

Que faire d'une vulnérabilité

- Presque toujours : mettre à jour l'image de base, reconstruire, redéployer.
- Bibliothèque de l'application : monter sa version.
- Paquet inutile : le retirer de l'image.

En faire une règle

- La chaîne d'intégration refuse de déployer une image qui présente une faille critique.
- Les images sont reconstruites régulièrement, même sans changement de code.

Limite. Le scan trouve les failles *connues* des composants *reconnus*. Il ne remplace ni la revue du code, ni la réduction de l'image.

Définition : tags immuables

Réglage d'un dépôt qui interdit de pousser une image sous un tag déjà utilisé. Un tag désigne alors une seule image, pour toujours.

Pourquoi

- `shop-api:1.4.2` désigne exactement ce qui a été testé.
- Un retour arrière vers une version précédente retrouve bien l'ancienne image.
- Personne ne peut écraser une image en production.

Définition : règle de cycle de vie ECR

Règle d'un dépôt qui supprime automatiquement des images selon leur nombre, leur âge ou leurs tags.

Sans elle, un dépôt alimenté par une chaîne d'intégration grossit de plusieurs images par jour, toutes facturées.

`{}` cycle.json : supprimer les images sans tag après 7 jours, garder les 20 dernières

```
{ "rules": [  
  { "rulePriority": 1,  
    "description": "Images sans tag : 7 jours",  
    "selection": {  
      "tagStatus": "untagged",  
      "countType": "sinceImagePushed",  
      "countUnit": "days",  
      "countNumber": 7 },  
    "action": { "type": "expire" } },  
  { "rulePriority": 2,  
    "description": "Garder les 20 plus recentes",  
    "selection": {  
      "tagStatus": "any",  
      "countType": "imageCountMoreThan",  
      "countNumber": 20 },  
    "action": { "type": "expire" } }  
]
```

\$ Appliquer la règle

```
aws ecr put-lifecycle-policy \  
  --repository-name user7/shop-api \  
  --lifecycle-policy-text file://cycle.json
```

Fonction	Usage
Politique IAM	Ce qu'une identité du compte peut faire : pousser, tirer, administrer.
Politique de dépôt	Autoriser un autre compte, ou toute l'organisation, à tirer les images.
Réplication	Copie automatique vers d'autres régions ou d'autres comptes.
Cache de registres externes (pull through cache)	ECR garde une copie des images tirées de Docker Hub ou d'autres registres publics.
VPC endpoints	Tirer des images depuis un sous-réseau sans sortie : <code>ecr.api</code> , <code>ecr.dkr</code> , et l'endpoint passerelle S3.

Poste	Tarif indicatif
Stockage	0,10 \$ par Go et par mois
Transfert vers un service de la même région	Gratuit
Transfert vers Internet ou une autre région	Facturé au gigaoctet
Scan avancé	Par image analysée

Organisation habituelle. Un compte central héberge les dépôts ; la chaîne d'intégration y pousse ; les comptes de test et de production y tirent, grâce à la politique des dépôts.

Droits minimaux. La chaîne d'intégration peut pousser ; les rôles d'exécution peuvent seulement tirer. La politique gérée `AmazonEC2ContainerRegistryReadOnly` couvre la lecture.

Erreur	Symptôme	Correction
Jeton expiré	« no basic auth credentials », « authorization token has expired ».	Refaire la connexion : le jeton dure 12 heures.
Dépôt inexistant	« repository does not exist ».	Créer le dépôt avant de pousser.
Image non renommée avec l'adresse du registre	Docker tente de pousser vers Docker Hub.	<code>docker tag</code> avec le nom complet.
Mauvaise région dans l'adresse	Connexion réussie, envoi refusé.	Même région pour la connexion et le nom de l'image.
Tag réutilisé sur un dépôt immuable	« tag invalid: already exists ».	Nouveau tag à chaque construction.
Aucune règle de cycle de vie	Stockage qui croît sans fin.	Règle sur les images sans tag et sur le nombre.
Scan activé, résultats jamais lus	Faillies connues en production.	Blocage dans la chaîne, ou alerte.
Tâche dans un sous-réseau sans sortie ni endpoints	Image impossible à tirer.	NAT Gateway, ou endpoints ECR et S3.

Images et registre, partie 3

Architectures x86_64 et ARM64

Une image est construite pour un type de processeur. La construire sur un Mac récent et la lancer sur un serveur classique est l'erreur la plus fréquente du débutant.

- Deux familles de processeurs
- L'erreur « exec format error »
- Construire pour la bonne architecture
- Les images multi-architecture
- Choisir l'architecture d'exécution
- Synthèse : images et registre

Deux familles de processeurs

	x86_64	ARM64
Autres noms	amd64 , x64	arm64 , aarch64
Fabricants	Intel, AMD	AWS (Graviton), Apple (M1 et suivants)
Sur AWS	Types d'instance sans la lettre « g »	Types avec la lettre « g » : t4g , m7g
Postes de travail	La plupart des PC ; anciens Mac	Mac depuis fin 2020 ; certains PC récents
Nom Docker	linux/amd64	linux/arm64

Définition : architecture d'une image

Type de processeur pour lequel les programmes de l'image ont été compilés. Une image ne s'exécute que sur une machine de la même architecture.

Ce qui dépend de l'architecture

- L'image de base : il en existe une variante par architecture.
- Tout programme compilé : binaires, bibliothèques natives.
- Les extensions natives des langages interprétés : paquets Python ou Node contenant du code C.

Par défaut, `docker build` construit pour l'architecture de la machine qui construit. Sur un Mac Apple Silicon, c'est donc ARM64.

Schéma : l'erreur d'architecture



L'image a été tirée sans erreur, mais ne contient aucun programme pour ce processeur.

La tâche démarre, échoue aussitôt, et recommence.

Comment la reconnaître. La tâche passe en « STOPPED » quelques secondes après son lancement, et son journal ne contient qu'une ligne : `exec format error`, ou `exec /usr/local/bin/python: exec format error`.

Vérifier une image. `docker image inspect --format '{{.Architecture}}' shop-api:1.4.2` en local ; dans ECR, la console affiche l'architecture de chaque image.

\$ Prérequis : Docker avec buildx (inclus dans Docker Desktop) ; connexion à ECR établie

```
# Construire pour x86_64, quelle que soit la machine
docker build --platform linux/amd64 \
  -t $REGISTRE/user7/shop-api:1.4.2 .
```

```
# Construire pour ARM64
docker build --platform linux/arm64 \
  -t $REGISTRE/user7/shop-api:1.4.2-arm64 .
```

```
# Construire les deux et pousser une image
# multi-architecture sous un seul tag
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t $REGISTRE/user7/shop-api:1.4.3 --push .
```

```
# Vérifier les variantes présentes dans le registre
docker buildx imagetools inspect \
  $REGISTRE/user7/shop-api:1.4.3
```

Définition : émulation

Exécution, par traduction à la volée, de programmes compilés pour une autre architecture. Elle permet de construire une image ARM64 sur un PC, et l'inverse, au prix d'une construction plus lente.

Trois façons d'obtenir la bonne image

- **Préciser `--platform`** : simple, lent quand l'émulation intervient.
- **Construire sur une machine de la bonne architecture** : une chaîne d'intégration, de préférence.
- **Image multi-architecture** : la question ne se pose plus à l'exécution.

Règle. Toujours écrire `--platform` dans les commandes et les chaînes de construction : le résultat ne dépend plus de la machine de celui qui construit.

Schéma : l'image multi-architecture



Un seul nom, un seul tag : chaque machine reçoit sa variante.

Définition : image multi-architecture

Tag qui désigne un index, c'est-à-dire la liste de plusieurs variantes de la même image, une par architecture. Le client choisit de lui-même celle qui lui convient.

C'est ce que font les images officielles. `python:3.13-slim` est une image multi-architecture : c'est pourquoi le même Dockerfile se construit indifféremment sur un Mac ou un PC.

Choisir l'architecture d'exécution

	x86_64	ARM64 (Graviton)
Prix sur Fargate	Référence.	Environ 20 % de moins.
Performance	Référence.	Comparable, souvent meilleure par dollar.
Compatibilité	Tout fonctionne.	Quelques logiciels anciens ou propriétaires n'existent pas en ARM64.
Poste Mac récent	Construction par émulation.	Construction native.

`{}` Dans la task definition ECS : l'architecture des tâches

```
"runtimePlatform": {
  "cpuArchitecture": "ARM64",
  "operatingSystemFamily": "LINUX"
}
```

Valeurs possibles. `x86_64` , celle par défaut, ou `ARM64` . L'image désignée par la task definition doit contenir la variante correspondante.

Démarche. Commencer en `x86_64`, où tout fonctionne. Publier des images multi-architecture. Passer en `ARM64` quand l'application est validée dessus : c'est une ligne à changer, pour une économie durable.

Ce qu'il faut retenir

- Une image est un modèle en couches, en lecture seule ; un conteneur en est une exécution, jetable.
- Le Dockerfile décrit la construction ; son ordre détermine l'efficacité du cache.
- Petite image, version précise, utilisateur non privilégié, aucun secret.
- ECR est le registre privé d'AWS ; on s'y connecte avec un jeton de 12 heures.
- Scan à l'envoi, tags immuables, règle de cycle de vie : trois réglages à activer sur chaque dépôt.
- Une image a une architecture ; `--platform` la fixe à la construction.
- Une image multi-architecture convient aux deux familles de processeurs.

1. Quelle différence entre une image et un conteneur ?

2. Pourquoi copier le fichier de dépendances avant le code ?

3. Combien de temps dure le jeton de connexion à ECR ?

4. Que garantit un dépôt aux tags immuables ?

5. À quoi sert une règle de cycle de vie ECR ?

6. Que signifie « exec format error » ?

7. Comment construire pour x86_64 depuis un Mac récent ?

Cliquer sur une question pour afficher la réponse.

ECS sur Fargate, partie 1

Amazon ECS

Dire à AWS quels conteneurs faire tourner, en combien d'exemplaires, et le laisser s'en charger.

- L'orchestration de conteneurs
- Cluster, task definition, tâche, service
- Contenu d'une task definition
- Le réseau d'une tâche
- Tailles de tâches
- Création

Définition : orchestrateur de conteneurs

Système qui lance les conteneurs sur un ensemble de machines, les surveille, les remplace quand ils tombent, les relie au réseau et en ajuste le nombre.

Définition : Amazon ECS (Elastic Container Service)

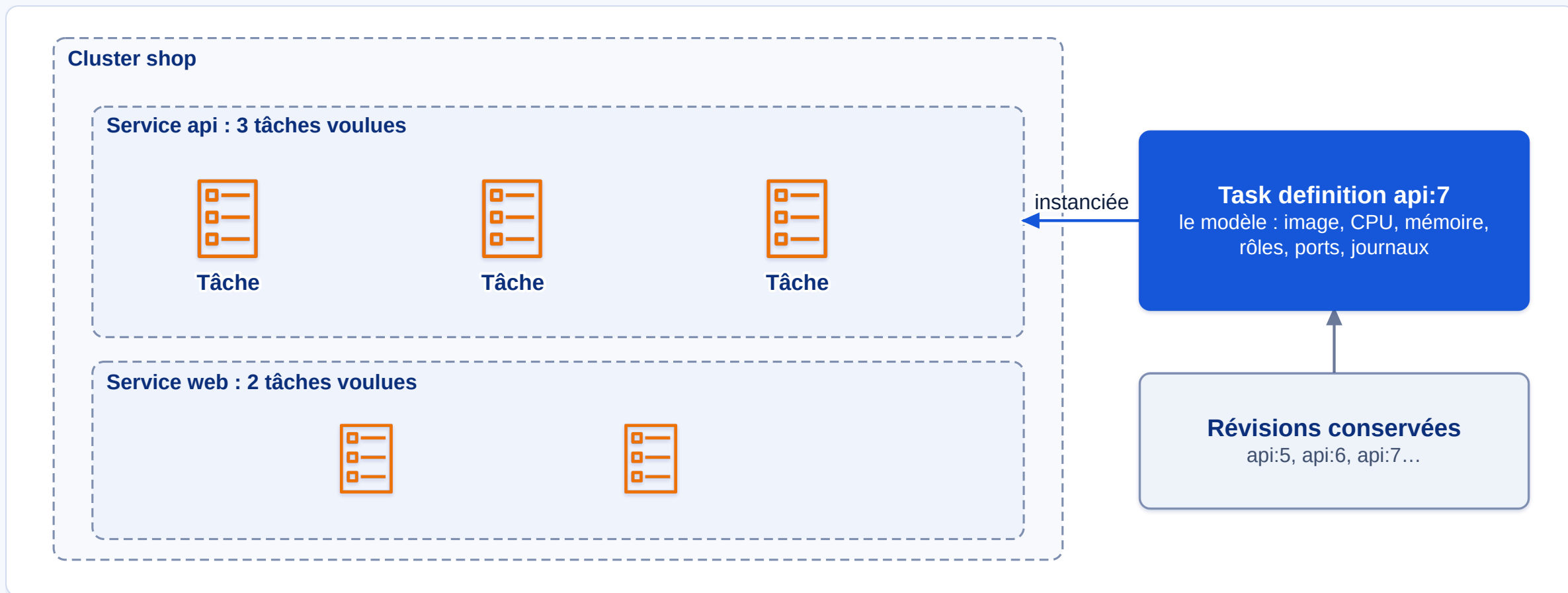
Orchestrateur de conteneurs conçu par AWS. On lui décrit des tâches et des services ; il les place, les maintient et les relie aux autres services AWS.

Ce qu'un orchestrateur ajoute à `docker run`. Plusieurs machines, plusieurs zones, le remplacement automatique, les déploiements sans coupure, l'intégration au load balancer et aux droits IAM.

Caractéristique d'ECS	Détail
Managé	Aucun plan de contrôle à installer ni à mettre à jour.
Gratuit	On ne paie que les ressources qui font tourner les conteneurs.
Intégré	IAM, VPC, load balancers, CloudWatch, Secrets Manager, Auto Scaling.
Simple	Quatre notions suffisent.
Propre à AWS	Ses définitions ne se réutilisent pas ailleurs.

Les quatre notions. Le **cluster**, où cela tourne ; la **task definition**, quoi lancer ; la **tâche**, ce qui tourne ; le **service**, combien et pour combien de temps.

Schéma : cluster, service, tâche, task definition

**Définition : cluster ECS**

Regroupement logique de services et de tâches, avec la capacité qui les fait tourner. Avec Fargate, il ne contient aucune machine à gérer.

Parallèle avec le chapitre « Élasticité ». La task definition joue le rôle du modèle de lancement ; le service, celui de l'Auto Scaling group ; la tâche, celui de l'instance. Les idées sont les mêmes, appliquées à des conteneurs.

Définition : task definition

Document versionné qui décrit une tâche : les conteneurs à lancer, leur image, leurs ressources, leurs ports, leurs variables, leurs journaux et les rôles IAM à utiliser.

Élément	Contenu
Famille et révision	<code>shop-api:7</code> : un nom, et un numéro qui augmente à chaque enregistrement.
CPU et mémoire	Pour toute la tâche : <code>256</code> unités (un quart de vCPU), <code>512</code> Mo.
Mode réseau	<code>awsvpc</code> , obligatoire avec Fargate.
Plateforme	Architecture et système : <code>X86_64</code> ou <code>ARM64</code> , Linux.
Rôle d'exécution	Pour tirer l'image et écrire les journaux.
Rôle de tâche	Pour les appels AWS de l'application.
Conteneurs	Un ou plusieurs, décrits un à un.

Par conteneur	Contenu
Image	Nom complet, avec tag ou empreinte.
Ports	Le port d'écoute du conteneur.
Variables	En clair, pour la configuration.
Secrets	Références à Secrets Manager ou Parameter Store.
Journaux	Pilote et destination.
Contrôle de santé	Commande exécutée dans le conteneur.
Essentiel	Si ce conteneur s'arrête, toute la tâche s'arrête.

Immuable. Une révision ne se modifie pas : tout changement en crée une nouvelle. Déployer, c'est faire passer un service d'une révision à la suivante ; revenir en arrière, à la précédente.

`taskdef.json` : l'API de UtopiosShop sur Fargate (remplacer le numéro de compte et user7)

```
{
  "family": "user7-shop-api",
  "requiresCompatibilities": ["FARGATE"],
  "networkMode": "awsvpc",
  "cpu": "256",
  "memory": "512",
  "runtimePlatform": { "cpuArchitecture": "X86_64", "operatingSystemFamily": "LINUX" },
  "executionRoleArn": "arn:aws:iam::123456789012:role/user7-ecs-execution",
  "taskRoleArn": "arn:aws:iam::123456789012:role/user7-shop-api-task",
  "containerDefinitions": [ {
    "name": "api",
    "image": "123456789012.dkr.ecr.eu-west-3.amazonaws.com/user7/shop-api:1.4.2",
    "essential": true,
    "portMappings": [ { "containerPort": 8080, "protocol": "tcp" } ],
    "environment": [ { "name": "LOG_LEVEL", "value": "info" } ],
    "secrets": [ { "name": "DB_SECRET",
      "valueFrom": "arn:aws:secretsmanager:eu-west-3:123456789012:secret:shop/prod/db-AbCdEf" } ],
    "logConfiguration": { "logDriver": "awslogs",
      "options": { "awslogs-group": "/ecs/user7-shop-api", "awslogs-region": "eu-west-3",
        "awslogs-stream-prefix": "api" } }
  } ]
}
```

Lecture. Une tâche d'un quart de vCPU et 512 Mo, avec un seul conteneur qui écoute sur le port 8080, reçoit un secret au démarrage et envoie ses journaux dans CloudWatch Logs. Les deux rôles sont détaillés dans la suite du chapitre.

Définition : tâche (task)

Exécution d'une task definition : un ou plusieurs conteneurs lancés ensemble, qui partagent le même réseau et la même durée de vie.

Définition : service ECS

Objet qui maintient en permanence un nombre voulu de tâches d'une task definition, les répartit entre les zones, les remplace si elles s'arrêtent, et les inscrit auprès d'un load balancer.

	Tâche seule	Service
Durée	S'exécute, puis s'arrête.	Tourne indéfiniment.
Si elle s'arrête	Rien : c'est terminé.	Une autre est lancée.
Usage	Traitement par lots, migration de base, tâche planifiée.	API, site, tout ce qui doit rester disponible.
Lancement	run-task	create-service

Plusieurs conteneurs dans une tâche. Possible, pour un conteneur d'accompagnement : collecte de journaux, proxy. Les conteneurs qui évoluent et se dimensionnent séparément vont dans des tâches, donc des services, distincts.

Le service est l'équivalent de l'Auto Scaling group : un nombre voulu, un remplacement automatique, une répartition entre zones. À la différence d'une instance, une tâche de remplacement démarre en quelques dizaines de secondes.

Amazon ECS

Schéma : le cycle de vie d'une tâche



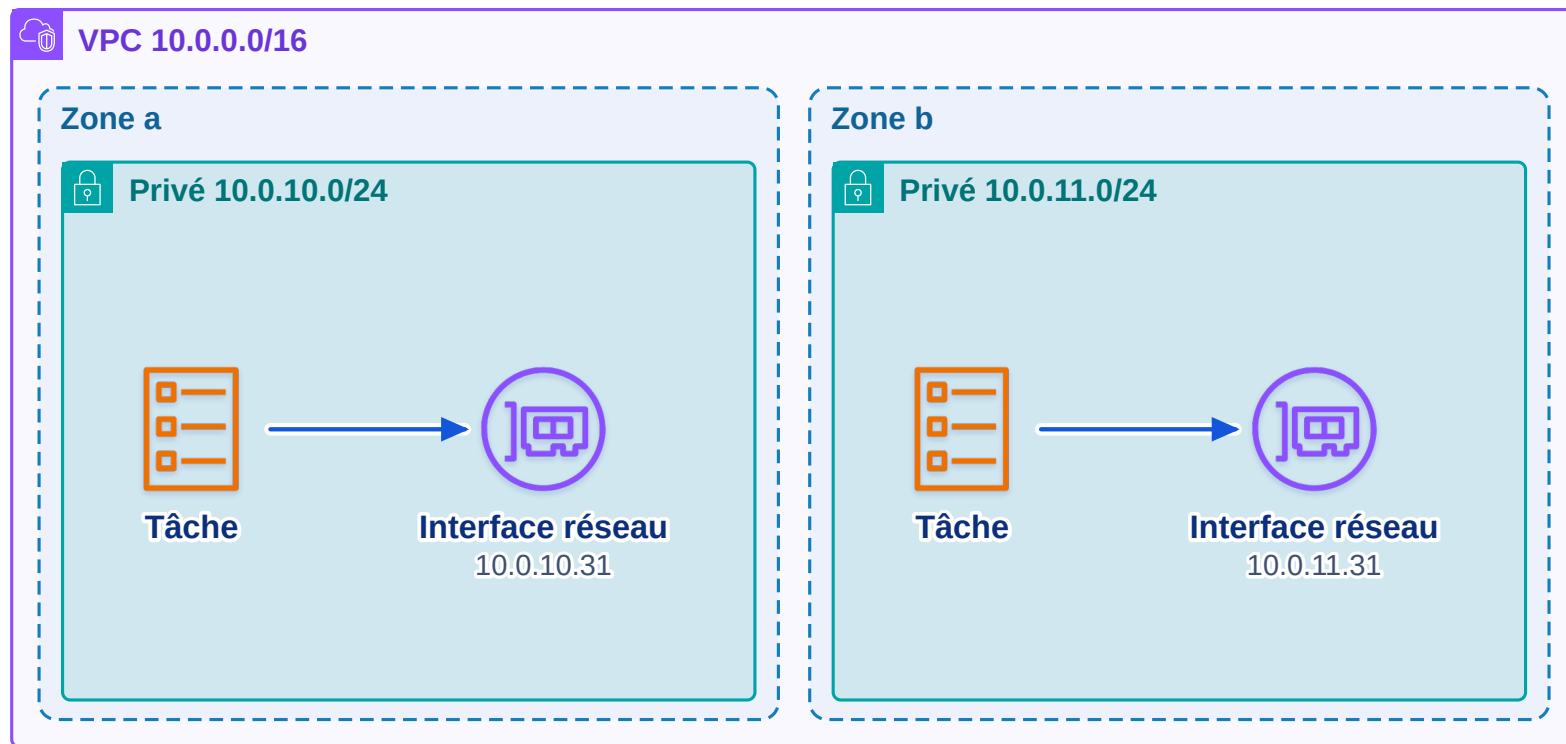
Une tâche ne redémarre jamais : arrêtée, elle est remplacée par une nouvelle.

C'est le service qui veille à ce que le nombre voulu de tâches soit toujours en marche.

Motif d'arrêt fréquent	Signification
Essential container in task exited	Le conteneur principal s'est arrêté : lire ses journaux.
CannotPullContainerError	Image introuvable, droits ou réseau manquants.
Task failed ELB health checks	Le load balancer la juge défaillante.
OutOfMemoryError	Mémoire de la tâche dépassée.

Premier réflexe de diagnostic. Console ECS, onglet des tâches, filtre « Stopped » : chaque tâche arrêtée affiche son motif et le code de sortie de ses conteneurs. Les tâches arrêtées ne restent visibles qu'environ une heure.

Schéma : le réseau d'une tâche



sg-api

8080 depuis sg-alb

Une tâche, une interface

sa propre adresse privée,
son propre security group

Définition : mode réseau awsvpc

Mode dans lequel chaque tâche reçoit sa propre interface réseau dans un sous-réseau du VPC, avec une adresse privée et ses propres security groups.

Conséquence. Tout ce qui a été vu sur le réseau s'applique tel quel aux tâches : sous-réseaux privés, chaînage de security groups, endpoints, Flow Logs. Une tâche se filtre comme une instance.

Dimensionner une tâche Fargate

CPU	Mémoire possible
0,25 vCPU (256)	0,5, 1 ou 2 Go
0,5 vCPU (512)	1 à 4 Go
1 vCPU (1024)	2 à 8 Go
2 vCPU (2048)	4 à 16 Go
4 vCPU (4096)	8 à 30 Go
8 et 16 vCPU	Jusqu'à 120 Go

Règles

- CPU et mémoire se fixent pour la **tâche** ; seules certaines combinaisons existent.
- Dépasser la mémoire arrête la tâche. Dépasser le CPU la ralentit seulement.
- Stockage éphémère : 20 Go inclus, extensible ; perdu à l'arrêt de la tâche.

Plusieurs petites tâches plutôt qu'une grosse. Quatre tâches d'un demi-vCPU réparties sur deux zones résistent mieux et s'ajustent plus finement qu'une tâche de 2 vCPU.

Mesurer. Les métriques d'utilisation du processeur et de la mémoire du service, dans CloudWatch, disent si la taille choisie est juste. **Container Insights** en donne le détail par tâche et par conteneur.

Ligne de commande : cluster, task definition, service

\$ Prérequis : image poussée dans ECR ; rôles créés (partie suivante) ; taskdef.json ; sous-réseaux privés PRIV_A, PRIV_B ; security group SG_API ; remplacer user7

1. Le cluster : avec Fargate, un simple nom

```
aws ecs create-cluster --cluster-name user7-shop
```

2. Le groupe de journaux attendu par la task definition

```
aws logs create-log-group --log-group-name /ecs/user7-shop-api
```

```
aws logs put-retention-policy --log-group-name /ecs/user7-shop-api --retention-in-days 14
```

3. Enregistrer la task definition : une nouvelle révision à chaque appel

```
aws ecs register-task-definition --cli-input-json file://taskdef.json \
  --query 'taskDefinition.[family,revision]' --output text
```

4. Créer le service : 2 tâches, sur Fargate, dans les sous-réseaux privés, sans adresse publique

```
aws ecs create-service --cluster user7-shop --service-name api \
  --task-definition user7-shop-api --desired-count 2 --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=[$PRIV_A,$PRIV_B],securityGroups=[$SG_API],assignPublicIp=DISABLED}"
```

5. Attendre la stabilité, puis lire l'état du service et de ses tâches

```
aws ecs wait services-stable --cluster user7-shop --services api
```

```
aws ecs describe-services --cluster user7-shop --services api \
```

```
  --query 'services[0].[status,desiredCount,runningCount,taskDefinition]' --output text
```

```
aws ecs list-tasks --cluster user7-shop --service-name api --query taskArns --output text
```

Sans numéro de révision, `--task-definition user7-shop-api` désigne la dernière révision active. En production, on écrit la révision explicitement : `user7-shop-api:7`.

Erreur	Symptôme	Correction
Tâche dans un sous-réseau privé sans NAT ni endpoints	<code>CannotPullContainerError</code> , délai dépassé.	NAT Gateway, ou endpoints ECR, S3 et CloudWatch Logs.
Tâche dans un sous-réseau public sans adresse publique	Même erreur.	<code>assignPublicIp=ENABLED</code> , ou mieux, sous-réseau privé.
Combinaison CPU et mémoire inexistante	Enregistrement refusé.	Consulter le tableau des tailles.
Groupe de journaux absent	<code>ResourceInitializationError</code> .	Le créer, ou autoriser sa création automatique.
Mémoire trop juste	Tâches arrêtées : <code>OutOfMemoryError</code> .	Augmenter la mémoire ; chercher une fuite.
Tag <code>latest</code> dans la task definition	Version en cours inconnue ; redéploiement sans effet visible.	Tag de version, ou empreinte.
Tâche lancée seule pour un service permanent	Pas de remplacement à l'arrêt.	Créer un service.
Tâche qui redémarre en boucle	Le service relance sans fin.	Lire le motif d'arrêt et les journaux ; activer le disjoncteur.

ECS sur Fargate, partie 2

Fargate ou EC2

Les conteneurs tournent toujours sur des machines. La question est de savoir qui s'en occupe.

- AWS Fargate
- Qui gère quoi
- Comparaison
- Ce que coûte Fargate
- Fargate Spot
- Quand préférer EC2

Définition : AWS Fargate

Moteur d'exécution de conteneurs sans serveur. On indique le processeur et la mémoire d'une tâche ; AWS fournit, isole et entretient la machine qui la fait tourner, sans jamais la montrer.

Propriété	Détail
Aucune instance	Ni AML, ni correctif, ni agent, ni dimensionnement de cluster.
Isolation	Chaque tâche tourne dans son propre environnement, qu'elle ne partage avec aucune autre.
Facturation	À la seconde, pour le processeur et la mémoire demandés, du téléchargement de l'image à l'arrêt.
Réseau	Une interface par tâche, dans vos sous-réseaux.
Disponible pour	ECS et EKS.

Ce que l'on gagne

- Plus de flotte à exploiter : tout le chapitre « L'instance EC2 » disparaît.
- La capacité est toujours exactement celle des tâches : rien de réservé pour rien.
- Le système hôte est tenu à jour par AWS.

Définition : type de lancement (launch type)

Façon dont ECS obtient la capacité pour une tâche : `FARGATE`, ou `EC2` sur des instances inscrites dans le cluster.

Le choix par défaut. Sauf besoin particulier, on lance ses conteneurs sur Fargate : c'est ce que fait cette formation.

Schéma : qui gère les machines

ECS sur instances EC2

Application et image

Task definition, service

Capacité du cluster : combien de machines

Instances : AMI, correctifs, agent ECS

Matériel, réseau, bâtiment

à votre charge

ECS sur Fargate

Application et image

Task definition, service

Capacité du cluster : combien de machines

Instances : AMI, correctifs, agent ECS

Matériel, réseau, bâtiment

géré par AWS

Lecture. Sur EC2, vous exploitez un Auto Scaling group d'instances en plus de vos conteneurs : deux niveaux à dimensionner, à corriger, à surveiller. Sur Fargate, il ne reste que l'application et sa description.

Troisième voie. Avec les « instances gérées » d'ECS, AWS choisit, lance et entretient des instances EC2 pour vous : on garde l'accès aux types d'instance, GPU compris, sans en assurer l'exploitation.

Fargate ou EC2

Fargate et EC2 comparés

	ECS sur Fargate	ECS sur instances EC2
Machines à gérer	Aucune.	Les instances du cluster.
Mise à l'échelle	Celle du service seulement.	Celle du service, et celle du cluster.
Démarrage d'une tâche	Quelques dizaines de secondes.	Quelques secondes si une instance a de la place ; minutes sinon.
Prix à ressource égale	Plus élevé.	Plus bas, si les instances sont bien remplies.
Gaspillage	Aucun : on paie la taille de la tâche.	La place inoccupée sur les instances.
GPU	Non.	Oui.
Accès à l'hôte, mode privilégié	Non.	Oui.
Engagement de prix	Compute Savings Plans.	Savings Plans, instances réservées, Spot.
Sécurité de l'hôte	Assurée par AWS.	À votre charge.

Changer d'avis. La même task definition peut être compatible avec les deux. Passer un service de l'un à l'autre est une modification de configuration, pas une réécriture.

Ce que coûte Fargate

Ressource	x86_64	ARM64
vCPU, par heure	0,0405 \$	0,0324 \$
Mémoire, par Go et par heure	0,0044 \$	0,0036 \$
Stockage éphémère au-delà de 20 Go	Facturé au gigaoctet	

Une tâche, un mois complet	Ordre de grandeur
0,25 vCPU, 0,5 Go	9 \$
0,5 vCPU, 1 Go	18 \$
1 vCPU, 2 Go	36 \$
Service de 2 tâches de 0,5 vCPU et 1 Go	36 \$

Ce qui s'ajoute

- Le load balancer.
- La NAT Gateway, ou les VPC endpoints, pour tirer les images.
- Les journaux dans CloudWatch Logs.
- Le stockage des images dans ECR.

Réduire la facture

- Dimensionner les tâches d'après les mesures.
- Architecture ARM64 : environ 20 % de moins.
- Compute Savings Plan sur la base permanente.
- Fargate Spot pour ce qui tolère l'interruption.
- Arrêter les environnements de test hors des heures de travail : nombre voulu à zéro.

Définition : Fargate Spot

Capacité Fargate inutilisée, proposée jusqu'à 70 % moins cher, qu'AWS peut reprendre avec un préavis de deux minutes.

Définition : fournisseur de capacité (capacity provider)

Réglage qui dit à ECS d'où tirer la capacité d'un service, et dans quelles proportions : `FARGATE`, `FARGATE_SPOT`, ou un groupe d'instances.

\$ Un service à base garantie, complété en Spot

```
RESEAU="subnets=[$PRIV_A,$PRIV_B],securityGroups=[$SG_API]"
aws ecs create-service --cluster user7-shop \
  --service-name worker \
  --task-definition user7-shop-worker --desired-count 6 \
  --capacity-provider-strategy \
    capacityProvider=FARGATE,base=2,weight=1 \
    capacityProvider=FARGATE_SPOT,weight=3 \
  --network-configuration "awsvpcConfiguration={$RESEAU}"
```

Lecture de la stratégie

- `base=2` : les deux premières tâches sont toujours sur Fargate ordinaire.
- Au-delà, une tâche sur quatre en Fargate, trois sur quatre en Spot.

Convient à Spot

Traitements par lots, files de travaux.

Services sans état, largement répliqués.

Environnements de test.

Ne convient pas

Tâche unique dont l'arrêt coupe le service.

Traitement long impossible à reprendre.

Charge soumise à un délai strict.

Même principe que les instances Spot du chapitre « L'instance EC2 » : l'application doit réagir au signal d'arrêt et terminer proprement en moins de deux minutes.

Quand préférer EC2, et les limites de Fargate

Besoin	Réponse
Cartes graphiques	EC2, ou instances gérées.
Conteneur privilégié, accès au noyau ou au matériel	EC2.
Très grand parc, taux de remplissage élevé et constant	EC2 peut revenir moins cher.
Démarrage quasi instantané de nombreuses tâches	EC2 avec de la capacité en réserve.
Agent à installer sur chaque machine	EC2 ; ou conteneur d'accompagnement sur Fargate.
Tâche de plus de 16 vCPU ou 120 Go	EC2.

Limite de Fargate	Détail
Stockage local	Éphémère ; 20 Go inclus, 200 Go au plus.
Stockage persistant	Par EFS, ou par un volume EBS attaché à la tâche.
Mode réseau	<code>awsvpc</code> uniquement.
Connexion à l'hôte	Impossible ; ECS Exec permet d'entrer dans le conteneur.
Cache d'images	Aucun entre deux tâches : l'image est tirée à chaque lancement.

Conséquence de la dernière ligne. Sur Fargate, la taille de l'image se paie à chaque démarrage, en secondes. Une raison de plus de soigner ses images.

ECS sur Fargate, partie 3

Rôles et journaux

Une tâche porte deux rôles, que l'on confond presque toujours la première fois : l'un sert à la démarrer, l'autre lui sert pendant qu'elle tourne.

- Deux rôles
- Le rôle d'exécution
- Le rôle de tâche
- Injecter des secrets
- Les journaux vers CloudWatch
- Diagnostiquer

Schéma : deux rôles, deux moments

**Définition : rôle d'exécution (execution role)**

Rôle qu'ECS endosse pour préparer la tâche : tirer l'image dans ECR, créer les flux de journaux, lire les secrets à injecter. L'application ne s'en sert jamais.

Définition : rôle de tâche (task role)

Rôle dont les identifiants temporaires sont remis à l'application, dans le conteneur, pour ses propres appels aux services AWS.

Rôle d'exécution et rôle de tâche

	Rôle d'exécution	Rôle de tâche
Utilisé par	L'agent ECS, c'est-à-dire la plateforme.	Le code de l'application.
Moment	Avant et pendant le démarrage.	Pendant toute l'exécution.
Droits typiques	Tirer l'image ; écrire les journaux ; lire les secrets référencés.	Lire un bucket, écrire dans une table, publier un message.
Politique habituelle	La politique gérée <code>AmazonECSTaskExecutionRolePolicy</code> .	Une politique écrite pour l'application.
Partage	Souvent commun à plusieurs services.	Un par application.
Obligatoire	Oui sur Fargate, dès qu'il y a ECR ou des journaux.	Seulement si l'application appelle AWS.
S'il manque un droit	La tâche ne démarre pas.	La tâche démarre ; l'application reçoit « AccessDenied ».

Même relation d'approbation. Les deux rôles sont endossés par le service `ecs-tasks.amazonaws.com` . Ce qui les distingue, ce sont leurs permissions, et le champ de la task definition où on les inscrit.

Moyen de s'en souvenir. Le rôle d'exécution sert à *faire démarrer* le conteneur. Le rôle de tâche sert *au* conteneur. Le second est l'équivalent du rôle d'instance du chapitre « L'instance EC2 ».

\$ Prérequis : droits iam:CreateRole et iam:PassRole ; remplacer user7

1. Relation d'approbation commune : le service des tâches ECS peut endosser ces rôles

```
cat > ecs-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Principal":{"Service":"ecs-tasks.amazonaws.com"},"Action":["sts:AssumeRole"]}]}
EOF
```

2. Rôle d'exécution : tirer l'image dans ECR, écrire dans CloudWatch Logs

```
aws iam create-role --role-name user7-ecs-execution --assume-role-policy-document file://ecs-trust.json
aws iam attach-role-policy --role-name user7-ecs-execution \
  --policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy
```

3. Rôle de tâche : ce dont l'application a besoin, et rien d'autre (ici, lire un bucket)

```
aws iam create-role --role-name user7-shop-api-task --assume-role-policy-document file://ecs-trust.json
cat > api-perm.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Action":["s3:GetObject","s3:ListBucket"],
  "Resource":["arn:aws:s3:::user7-shop-images","arn:aws:s3:::user7-shop-images/*"]}]}
EOF
aws iam put-role-policy --role-name user7-shop-api-task --policy-name lecture-images --policy-document file://api-perm.json
```

4. Vérifier, depuis un conteneur en marche (par ECS Exec) : quelle identité l'application porte-t-elle ?

```
# aws sts get-caller-identity → .../assumed-role/user7-shop-api-task/...
```

Droit de celui qui déploie. Enregistrer une task definition ou créer un service qui cite ces rôles demande `iam:PassRole` sur chacun d'eux : c'est le garde-fou vu au chapitre « La CLI et les rôles ».

`{ }` Dans la définition du conteneur : variables en clair, et secrets par référence

```
"environment": [ { "name": "LOG_LEVEL", "value": "info" } ],
"secrets": [
  { "name": "DB_PASSWORD",      "valueFrom": "arn:aws:secretsmanager:eu-west-3:123456789012:secret:shop/prod/db-AbCdEf:password:" },
  { "name": "API_PARTENAIRE", "valueFrom": "arn:aws:ssm:eu-west-3:123456789012:parameter/shop/prod/api/cle" }
]
```

	environment	secrets
Valeur	Écrite en clair dans la task definition.	Une référence ; la valeur reste dans Secrets Manager ou Parameter Store.
Lue par	Personne : elle est déjà là.	ECS, avec le rôle d'exécution, au démarrage.
Vue dans la console	Oui.	Non : seule la référence apparaît.

Droits à ajouter au rôle d'exécution.
secretsmanager:GetSecretValue ou ssm:GetParameters sur les secrets cités, et kms:Decrypt si la clé est gérée par le client.

La valeur arrive en variable d'environnement du conteneur. Le suffixe :password: extrait un seul champ d'un secret au format JSON.

Rotation. La valeur est lue une fois, au démarrage de la tâche. Après la rotation d'un secret, les tâches en cours gardent l'ancienne : il faut les renouveler, ou faire lire le secret par l'application elle-même, avec son rôle de tâche.

Schéma : les journaux vers CloudWatch Logs



L'application n'écrit dans aucun fichier : elle affiche, et la plateforme collecte.

`{ }` Dans la définition du conteneur

```
"logConfiguration": {
  "logDriver": "awslogs",
  "options": {
    "awslogs-group": "/ecs/user7-shop-api",
    "awslogs-region": "eu-west-3",
    "awslogs-stream-prefix": "api"
  }
}
```

Définition : pilote de journaux awslogs

Mécanisme d'ECS qui capture tout ce qu'un conteneur écrit sur sa sortie standard et sa sortie d'erreur, et l'envoie dans un groupe de journaux CloudWatch.

Fixer une rétention. Un groupe de journaux conserve tout, indéfiniment, par défaut. Sans durée de conservation, les journaux finissent par coûter plus cher que les tâches.

Lire les journaux et entrer dans un conteneur

\$ Lire les journaux

```
# Suivre en direct tous les flux du groupe
aws logs tail /ecs/user7-shop-api --follow

# Les 30 dernières minutes, lignes contenant ERROR
aws logs tail /ecs/user7-shop-api --since 30m \
  --filter-pattern ERROR

# Pourquoi la dernière tâche s'est-elle arrêtée ?
TACHE=$(aws ecs list-tasks --cluster user7-shop \
  --service-name api --desired-status STOPPED \
  --query 'taskArns[0]' --output text)
aws ecs describe-tasks --cluster user7-shop --tasks $TACHE \
  --query 'tasks[0].[stoppedReason,
    containers[0].exitCode,containers[0].reason]'
```

Définition : ECS Exec

Fonction qui ouvre un terminal dans un conteneur en marche, à travers Systems Manager, sans port ni clé. C'est l'équivalent de `docker exec`.

\$ Entrer dans un conteneur (service créé avec `--enable-execute-command`)

```
aws ecs execute-command --cluster user7-shop \
  --task $TACHE_EN_MARCHE --container api \
  --interactive --command "/bin/sh"
```

Conditions. Option activée sur le service, droits `ssmmessages` dans le rôle de tâche, plugin Session Manager sur le poste, et un interpréteur de commandes présent dans l'image.

Pour diagnostiquer, pas pour corriger. Toute modification faite dans un conteneur disparaît avec lui : la correction se fait dans l'image.

Message ou symptôme	Cause	Correction
<code>CannotPullContainerError ... AccessDenied</code>	Le rôle d'exécution ne peut pas tirer dans ECR.	Politique <code>AmazonECSTaskExecutionRolePolicy</code> .
<code>ResourceInitializationError: unable to pull secrets</code>	Droit manquant sur le secret ou la clé KMS, ou pas d'accès réseau.	Compléter le rôle d'exécution ; vérifier NAT ou endpoints.
<code>ResourceInitializationError ... log group does not exist</code>	Groupe de journaux absent.	Le créer avant le déploiement.
L'application reçoit « <code>AccessDenied</code> » sur S3	Droit ajouté au rôle d'exécution au lieu du rôle de tâche.	Le mettre dans le rôle de tâche.
« <code>Unable to locate credentials</code> » dans le conteneur	Aucun rôle de tâche dans la task definition.	Renseigner <code>taskRoleArn</code> .
<code>is not authorized to perform: iam:PassRole</code>	Celui qui déploie ne peut pas confier le rôle.	Lui accorder <code>iam:PassRole</code> sur ces rôles.
Aucun journal dans CloudWatch	L'application écrit dans un fichier.	Écrire sur la sortie standard.
Mot de passe en clair dans <code>environment</code>	Visible de tous dans la console.	Bloc <code>secrets</code> .

Départager en une question. La tâche a-t-elle démarré ? Si non, chercher du côté du rôle d'exécution, de l'image ou du réseau. Si oui, et que l'application échoue, chercher du côté du rôle de tâche.

Ce qu'il faut retenir

- ECS : un cluster, des task definitions versionnées, des tâches, et des services qui maintiennent le nombre voulu.
- Une révision de task definition ne se modifie pas : on en crée une nouvelle.
- Chaque tâche a sa propre interface réseau et son security group.
- Fargate : aucune machine à gérer, facturation à la taille de la tâche. Le choix par défaut.
- EC2 pour les GPU, le mode privilégié, ou un très grand parc bien rempli.
- Rôle d'exécution : pour démarrer la tâche. Rôle de tâche : pour l'application.
- Les journaux vont de la sortie standard à CloudWatch Logs, avec une rétention.

1. Quelle différence entre une tâche et un service ?

2. Peut-on modifier une révision de task definition ?

3. Que facture Fargate ?

4. Quel rôle sert à tirer l'image dans ECR ?

5. Quel rôle permet à l'application de lire un bucket ?

6. Quel service endosse ces deux rôles ?

7. Où l'application doit-elle écrire ses journaux ?

Cliquer sur une question pour afficher la réponse.

Exploiter un service ECS, partie 1

Exposer un service par un ALB

Les tâches naissent et meurent, leurs adresses changent sans cesse : le load balancer est le seul point fixe.

- Pourquoi un load balancer
- Le target group de type ip
- Contrôles de santé
- Security groups
- Création du service
- Plusieurs services, un load balancer

Pourquoi un load balancer devant un service

Chaque tâche a sa propre adresse privée, attribuée à son lancement et perdue à son arrêt. Après un déploiement ou une mise à l'échelle, plus aucune adresse n'est la même.

Ce que fait l'intégration

- Le service **inscrit** chaque nouvelle tâche dans le target group, dès qu'elle démarre.
- Le load balancer ne lui envoie du trafic qu'une fois sa santé vérifiée.
- Le service **désinscrit** la tâche avant de l'arrêter, et lui laisse finir ses requêtes.

Tout cela sans aucune action de votre part.

Rappel du chapitre « Répartition de charge »

Ici

Listener

HTTPS sur 443, avec un certificat ACM.

Règle

Par chemin ou par hôte, vers un target group.

Target group

De type `ip` : les cibles sont les adresses des tâches.

Contrôle de santé

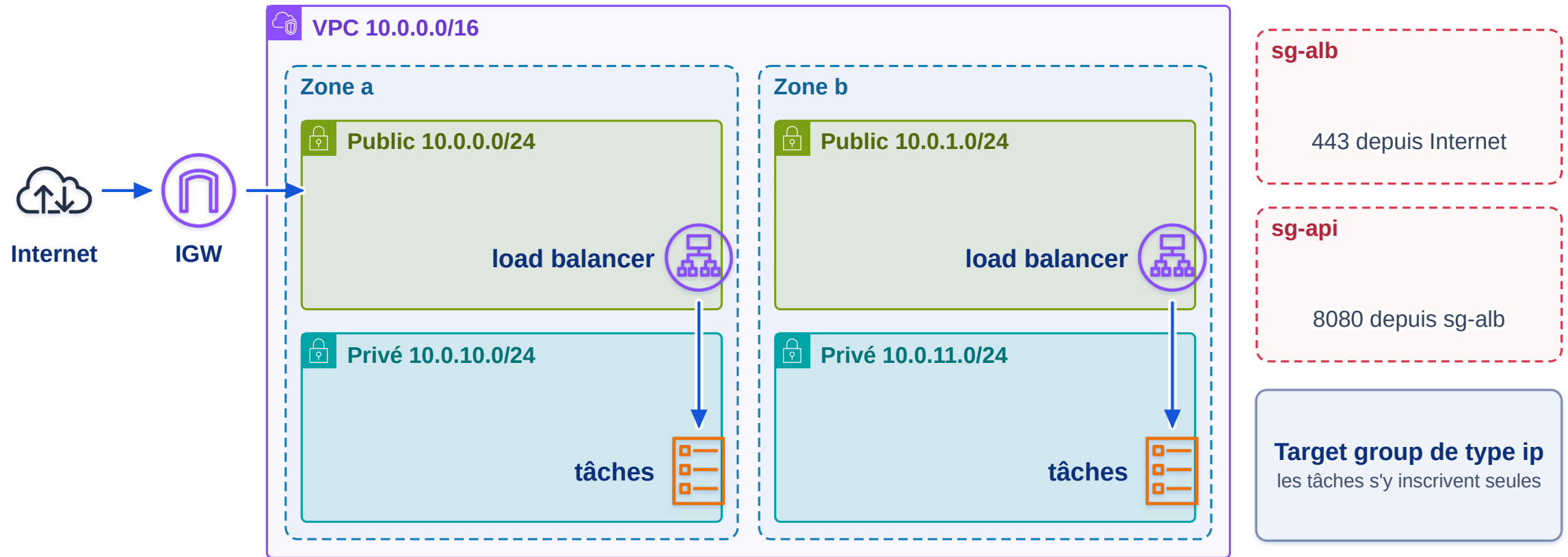
Sur un chemin de l'application.

Définition : target group de type ip

Target group dont les cibles sont des adresses IP plutôt que des instances. C'est le type requis pour des tâches en mode réseau `awsvpc`, donc pour Fargate.

Le lien se déclare à la création du service : target group, nom du conteneur et port. Il ne se change pas simplement ensuite ; mieux vaut le prévoir d'emblée.

Schéma : un service derrière un ALB



Lecture. Le load balancer occupe les sous-réseaux publics ; les tâches, les sous-réseaux privés, sans adresse publique. Le chaînage des security groups s'applique : `sg-api` n'accepte que `sg-alb`, sur le port du conteneur.

Deux zones au moins. Le service répartit ses tâches entre les sous-réseaux qu'on lui donne. Avec deux tâches sur deux zones, la perte d'une zone laisse le service debout, le temps que la tâche manquante soit relancée ailleurs.

Deux contrôles de santé, et trois délais

	Contrôle du target group	Contrôle du conteneur
Déclaré dans	Le target group.	La task definition.
Exécuté par	Le load balancer, par le réseau.	Une commande, dans le conteneur.
Vérifie	Que l'application répond à une requête HTTP.	Ce que la commande teste.
En cas d'échec	La tâche est remplacée.	La tâche est remplacée.

Délai	Rôle
Délai de grâce du service	Temps laissé à une tâche neuve avant que les échecs du load balancer comptent.
Délai de désinscription du target group	Temps laissé aux requêtes en cours quand une tâche est retirée. 300 s par défaut.
Délai d'arrêt du conteneur	Temps entre le signal d'arrêt et l'arrêt forcé. 30 s par défaut.

Le piège de la boucle. Une application qui met 40 secondes à démarrer, sans délai de grâce : le load balancer la juge défaillante, ECS la remplace, la suivante subit le même sort. Le délai de grâce doit couvrir le temps réel de démarrage.

Réglages de départ. Chemin de santé dédié et rapide ; intervalle de 10 secondes ; délai de grâce de 60 secondes ; délai de désinscription ramené à 30 secondes si les requêtes sont courtes.

\$ Prérequis : cluster et task definition existants ; ALB avec son listener ; VPC_ID, PRIV_A, PRIV_B, SG_ALB définis ; remplacer user7

1. Security group des tâches : le port du conteneur, depuis le load balancer uniquement

```
SG_API=$(aws ec2 create-security-group --group-name user7-sg-api --description "Taches de l API" \
--vpc-id $VPC_ID --query GroupId --output text)
```

```
aws ec2 authorize-security-group-ingress --group-id $SG_API --protocol tcp --port 8080 --source-group $SG_ALB
```

2. Target group de type ip, avec un contrôle de santé sur /health

```
TG=$(aws elbv2 create-target-group --name user7-tg-api --protocol HTTP --port 8080 --vpc-id $VPC_ID \
--target-type ip --health-check-path /health --health-check-interval-seconds 10 \
--healthy-threshold-count 2 --query 'TargetGroups[0].TargetGroupArn' --output text)
```

```
aws elbv2 modify-target-group-attributes --target-group-arn $TG \
--attributes Key=deregistration_delay.timeout_seconds,Value=30
```

3. Rattacher le target group au listener du load balancer (LISTENER : son ARN)

```
aws elbv2 modify-listener --listener-arn $LISTENER --default-actions Type=forward,TargetGroupArn=$TG
```

4. Créer le service, lié au target group

```
aws ecs create-service --cluster user7-shop --service-name api --task-definition user7-shop-api \
--desired-count 2 --launch-type FARGATE --health-check-grace-period-seconds 60 \
--network-configuration "awsvpcConfiguration={subnets=[$PRIV_A,$PRIV_B],securityGroups=[$SG_API],assignPublicIp=DISABLED}" \
--load-balancers "targetGroupArn=$TG,containerName=api,containerPort=8080"
```

5. Les tâches sont-elles inscrites et saines ?

```
aws elbv2 describe-target-health --target-group-arn $TG \
--query 'TargetHealthDescriptions[].[Target.Id,TargetHealth.State]' --output table
```

Ordre obligatoire. Le target group doit être rattaché à un listener avant la création du service, sans quoi ECS refuse le lien.

Schéma : plusieurs services derrière un load balancer



Lecture. Chaque service a son target group ; une règle du listener l'associe à un chemin ou à un nom d'hôte. Un seul load balancer, un seul certificat et un seul nom servent toute l'application.

Chaque service vit sa vie. Sa propre task definition, son propre nombre de tâches, ses propres déploiements et sa propre mise à l'échelle. C'est ce qui rend le découpage en services praticable.

Les services qui se parlent entre eux

Solution	Principe
Load balancer interne	Un second ALB, « internal », dans les sous-réseaux privés. Simple, déjà connu ; un coût fixe.
Service Connect	ECS donne à chaque service un nom court, et un proxy dans chaque tâche répartit les appels, réessaie et mesure.
Découverte par DNS	Chaque service obtient un nom dans une zone privée, qui renvoie les adresses de ses tâches.

Définition : découverte de services

Mécanisme par lequel un service trouve l'adresse d'un autre par son nom, sans configuration à la main, alors que ses instances changent en permanence.

Définition : ECS Service Connect

Fonction d'ECS qui relie les services d'un même espace de noms : chacun appelle l'autre par un nom simple, comme `http://paiement:8080`.

Security groups. Un service qui en appelle un autre : le security group de l'appelé accepte celui de l'appelant, sur son port. Toujours le même chaînage.

Exposition : erreurs fréquentes

Erreur	Symptôme	Correction
Target group de type <code>instance</code>	Création du service refusée.	Type <code>ip</code> .
<code>sg-api</code> n'autorise pas <code>sg-alb</code>	Cibles « unhealthy », erreurs 504, tâches remplacées en boucle.	Règle entrante depuis le security group du load balancer.
Port du target group différent du port du conteneur	Cibles « unhealthy ».	Aligner <code>containerPort</code> , target group et security group.
Délai de grâce trop court	Tâches saines tuées pendant leur démarrage.	L'allonger.
Chemin de santé qui interroge la base	Une lenteur de base fait remplacer toutes les tâches.	Contrôle limité à l'application elle-même.
Application qui écoute sur <code>127.0.0.1</code>	Contrôles de santé en échec.	Écouter sur <code>0.0.0.0</code> .
Target group non rattaché à un listener	Création du service refusée.	Créer la règle ou l'action d'abord.
Tâches avec une adresse publique	Load balancer contournable.	<code>assignPublicIp=DISABLED</code> , sous-réseaux privés.
Erreur 503 du load balancer	Aucune tâche inscrite.	Lire les événements du service.

Exploiter un service ECS, partie 2

Déployer : mise à jour progressive et retour arrière

Remplacer les tâches d'un service par celles de la nouvelle version, sans interruption, et revenir en arrière tout seul si elle ne démarre pas.

- Ce qu'est un déploiement
- La mise à jour progressive
- Minimum et maximum
- Le disjoncteur
- Le retour arrière
- Autres stratégies

Les trois étapes

1. Construire et pousser une nouvelle image, sous un nouveau tag.
 2. Enregistrer une nouvelle révision de la task definition, qui cite cette image.
 3. Mettre à jour le service pour qu'il utilise cette révision.
- À partir de là, ECS remplace les tâches lui-même.

Définition : déploiement

Passage d'un service d'une révision de task definition à une autre. ECS crée un déploiement, le fait progresser, et le déclare terminé ou en échec.

\$ Déployer une nouvelle révision

```
# Après avoir modifié l'image dans taskdef.json
aws ecs register-task-definition \
  --cli-input-json file://taskdef.json

aws ecs update-service --cluster user7-shop \
  --service api --task-definition user7-shop-api

aws ecs wait services-stable \
  --cluster user7-shop --services api
```

On déploie aussi une configuration. Changer une variable, une taille ou un rôle crée de même une nouvelle révision, et se déploie de la même façon.

Lien avec le chapitre « Élasticité ». Comme l'« instance refresh », un déploiement ne modifie rien : il remplace. Les tâches de l'ancienne révision sont arrêtées, pas mises à jour.

Déploiement

La mise à jour progressive

Définition : mise à jour progressive (rolling update)
Stratégie de déploiement par défaut d'ECS : les nouvelles tâches sont lancées et les anciennes retirées par étapes, en gardant à tout moment assez de tâches saines pour servir le trafic.

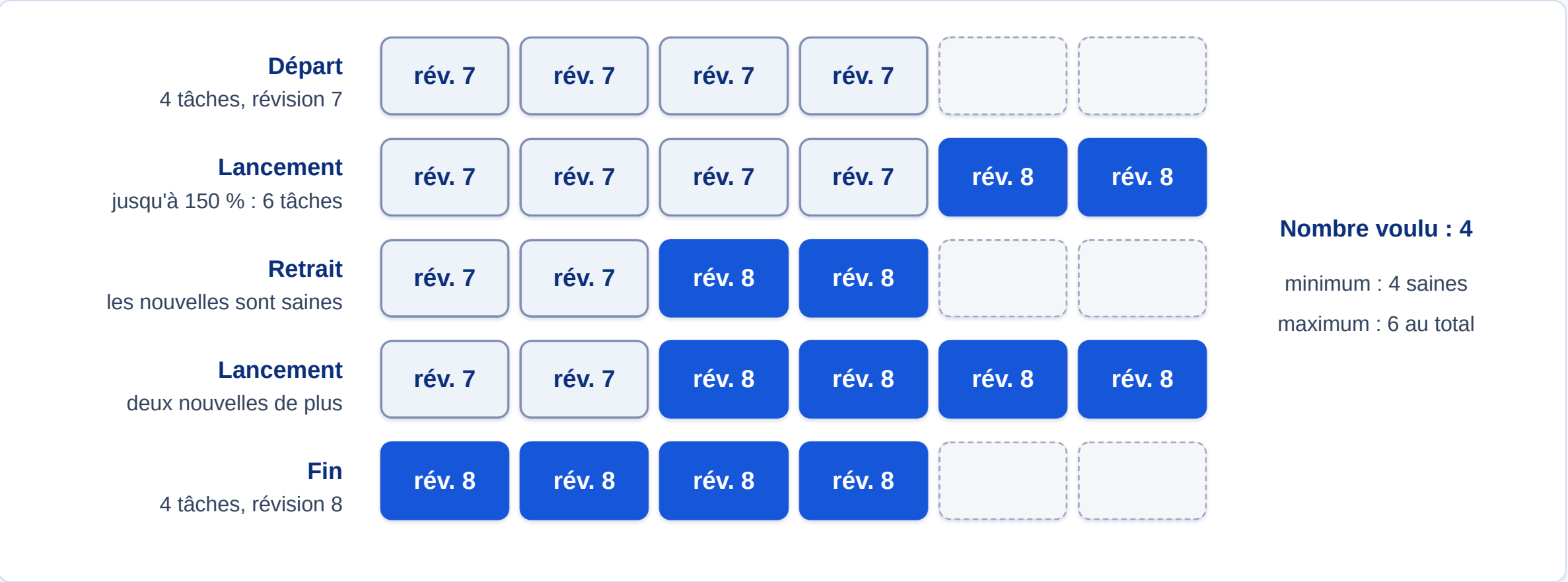
Définition : pourcentage minimal de tâches saines
Part du nombre voulu de tâches qui doit rester en service pendant le déploiement. À 100 %, la capacité ne baisse jamais.

Définition : pourcentage maximal
Part du nombre voulu jusqu'à laquelle le service peut monter pendant le déploiement. À 200 %, il peut lancer toutes les nouvelles tâches avant de retirer les anciennes.

Minimum	Maximum	Comportement, pour 4 tâches voulues
100 %	200 %	4 nouvelles lancées d'un coup, puis 4 anciennes retirées. Rapide, capacité doublée un instant.
100 %	150 %	Par lots de 2 : jamais moins de 4, jamais plus de 6.
50 %	100 %	2 anciennes retirées d'abord, puis remplacées. Aucune capacité en plus, mais moitié moins de tâches pendant le déploiement.

Valeurs par défaut. Minimum 100 %, maximum 200 % : le déploiement ne réduit jamais la capacité. Sur Fargate, où la capacité supplémentaire ne demande aucune machine, il n'y a guère de raison d'en changer.

Schéma : une mise à jour progressive



Lecture. Une ancienne tâche n'est retirée que lorsqu'une nouvelle est saine, c'est-à-dire démarrée et validée par le contrôle de santé du load balancer. Pendant tout le déploiement, les deux révisions servent du trafic en même temps.

Conséquence pour l'application. Deux versions coexistent quelques minutes : une nouvelle version doit rester compatible avec l'ancienne, notamment pour le schéma de la base et le format des messages.

Schéma : le disjoncteur de déploiement



Sans disjoncteur, ECS réessaie indéfiniment de lancer des tâches qui ne démarreront jamais.

Avec lui, l'échec est constaté en quelques minutes, et corrigé sans intervention.

Définition : disjoncteur de déploiement (deployment circuit breaker)

Mécanisme qui surveille un déploiement en cours et l'arrête quand trop de nouvelles tâches échouent. Avec l'option de retour arrière, il rétablit la dernière révision qui avait réussi.

\$ Activer le disjoncteur et le retour arrière sur un service

```
aws ecs update-service --cluster user7-shop \
  --service api --deployment-configuration '{
    "deploymentCircuitBreaker":
      { "enable": true, "rollback": true },
    "minimumHealthyPercent": 100,
    "maximumPercent": 200 }'
```

Ce que le disjoncteur détecte, et ce qu'il laisse passer

Déecté	Non détecté
Image introuvable ou impossible à tirer.	Erreur dans une fonction de l'application.
Conteneur qui s'arrête au démarrage.	Réponses fausses, mais avec un code 200.
Contrôle de santé qui échoue.	Lenteur, tant que le contrôle de santé répond.
Droit manquant dans le rôle d'exécution.	Droit manquant dans le rôle de tâche, si l'application démarre quand même.

Pour aller plus loin que le démarrage

- Associer au déploiement des **alarmes CloudWatch** : taux d'erreurs, latence. Si l'une se déclenche pendant le déploiement, ECS l'arrête et revient en arrière.
- Un contrôle de santé qui vérifie ce qui compte vraiment.

À activer sur tous les services. Disjoncteur et retour arrière ne coûtent rien et transforment un déploiement raté en non-événement : l'ancienne version n'a jamais cessé de servir.

Seuil. Le nombre d'échecs tolérés dépend de la taille du service, avec un minimum de 3. Sur un petit service, l'échec est constaté en quelques minutes.

\$ Prérequis : un déploiement en cours ou récent sur le service api

```
# 1. État du déploiement : PRIMARY est la révision visée ; rolloutState vaut IN_PROGRESS, COMPLETED ou FAILED
aws ecs describe-services --cluster user7-shop --services api --output table \
  --query 'services[0].deployments[][status,taskDefinition,desiredCount,runningCount,failedTasks,rolloutState]'

# 2. Le journal du service : ce qu'ECS a fait, et pourquoi
aws ecs describe-services --cluster user7-shop --services api \
  --query 'services[0].events[:8].[createdAt,message]' --output text

# 3. Quelles révisions existent ?
aws ecs list-task-definitions --family-prefix user7-shop-api --sort DESC --max-items 5

# 4. Retour arrière manuel : redéployer explicitement la révision précédente
aws ecs update-service --cluster user7-shop --service api --task-definition user7-shop-api:6

# 5. Relancer toutes les tâches sans changer de révision (pour relire un secret, par exemple)
aws ecs update-service --cluster user7-shop --service api --force-new-deployment
```

Les événements du service sont la première chose à lire quand un déploiement n'avance pas : « unable to place a task », « failed ELB health checks », « has reached a steady state ».

Le retour arrière suppose que l'ancienne image existe encore. Des tags immuables, et une règle de cycle de vie ECR qui garde assez de versions.

Stratégie	Principe	Intérêt
Progressive	Remplacement par lots, les deux versions mêlées.	Simple, sans capacité durablement doublée. Le choix par défaut.
Bleu/vert	La nouvelle version est lancée entièrement à côté de l'ancienne, testée, puis tout le trafic bascule d'un coup.	Test avant exposition ; retour arrière instantané.
Canari	Une petite part du trafic, par exemple 10 %, va d'abord à la nouvelle version ; le reste suit si tout va bien.	Un défaut ne touche qu'une fraction des utilisateurs.
Linéaire	Le trafic passe à la nouvelle version par paliers réguliers.	Montée en charge graduelle et observée.

Définition : déploiement bleu/vert

Stratégie dans laquelle deux environnements complets coexistent : l'ancien (bleu) et le nouveau (vert). On bascule le trafic de l'un à l'autre, et l'on garde l'ancien le temps de s'assurer que tout va bien.

Dans ECS. Ces stratégies sont prises en charge par le service lui-même, avec un load balancer ou Service Connect pour répartir le trafic entre les deux versions. On commence par la mise à jour progressive ; on y vient quand le coût d'un déploiement raté le justifie.

Erreur	Conséquence	Correction
Disjoncteur non activé	Déploiement raté qui tourne en boucle pendant des heures.	L'activer, avec retour arrière.
Même tag réutilisé pour chaque image	Révision inchangée : rien ne se déploie, ou une version inconnue.	Un tag par construction ; dépôt immuable.
Minimum à 0 % sur un service de production	Coupure pendant le déploiement.	Minimum à 100 %.
Nouvelle version incompatible avec l'ancienne	Erreurs pendant la coexistence.	Changements de schéma en deux temps.
Contrôle de santé trop indulgent	Version cassée déclarée saine, anciennes tâches retirées.	Contrôle significatif ; alarmes de déploiement.
Ancienne image supprimée	Retour arrière impossible.	Règle de cycle de vie qui garde les dernières versions.
Correction faite dans un conteneur par ECS Exec	Perdue au déploiement suivant.	Corriger l'image.
Déploiement lancé sans attendre le résultat	Échec découvert par les utilisateurs.	<code>wait services-stable</code> , ou suivi dans la chaîne.

Exploiter un service ECS, partie 3

L'auto scaling d'un service

Faire varier le nombre de tâches avec la charge : le suivi de cible du chapitre « Élasticité », appliqué aux conteneurs.

- Ce qui varie
- Application Auto Scaling
- Suivi de cible
- Mise en œuvre
- Fargate et EC2
- Synthèse : exploiter un service ECS

Auto scaling

Mettre à l'échelle un service ECS

Définition : Application Auto Scaling

Service qui fait varier automatiquement la capacité de ressources autres que des instances EC2 : nombre de tâches d'un service ECS, capacité d'une table DynamoDB, réplicas Aurora.

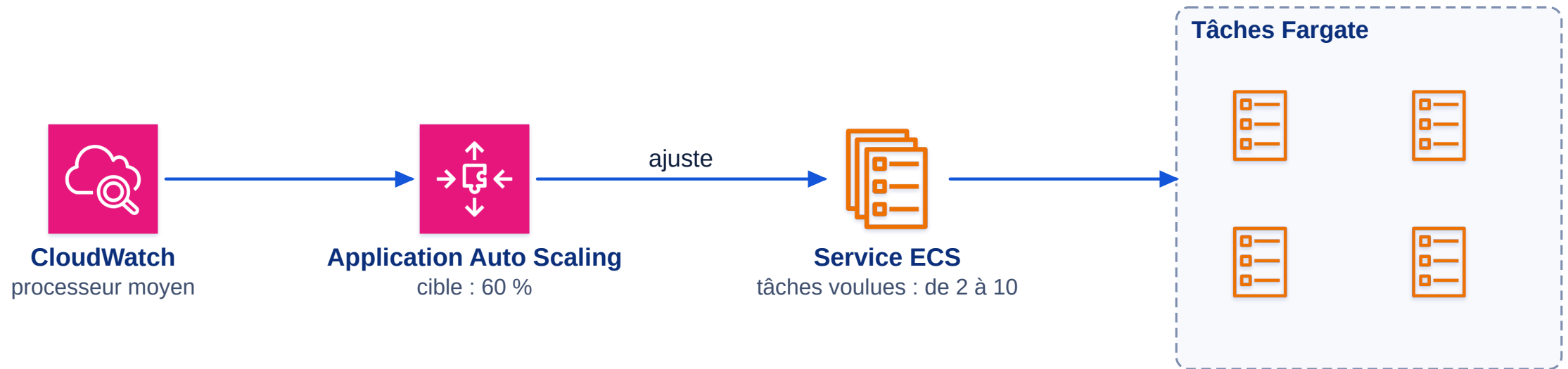
Élasticité (instances)	Ici (tâches)
Auto Scaling group	Service ECS
Capacité voulue	Nombre de tâches voulu
Minimum, maximum	Minimum, maximum
Suivi de cible	Suivi de cible
Action planifiée	Action planifiée
Minutes pour une instance	Dizaines de secondes pour une tâche

Métrique prédéfinie	Usage
Processeur moyen du service	Le cas général.
Mémoire moyenne du service	Application limitée par la mémoire.
Requêtes par cible du load balancer	Service web ou API : très bon indicateur.
Métrique personnalisée	Longueur d'une file de messages, pour des tâches de traitement.

Tout ce qui a été dit au chapitre « Élasticité » vaut ici : cible entre 40 et 60 %, ajout rapide et retrait prudent, minimum de deux tâches sur deux zones, maximum fixé par le budget.

L'avantage des conteneurs. Une tâche démarre bien plus vite qu'une instance : le service suit la charge de plus près, et la cible peut être un peu plus haute.

Schéma : l'auto scaling d'un service



Sur Fargate, ajouter une tâche suffit : il n'y a aucune machine à ajouter derrière.

Lecture. Application Auto Scaling observe une métrique du service, la compare à la cible, et modifie le nombre de tâches voulu, entre le minimum et le maximum. Le service lance ou retire alors des tâches, qui s'inscrivent ou se désinscrivent du load balancer.

Prérequis de l'élasticité. Comme pour les instances : des tâches sans état. Sessions dans un cache, fichiers dans S3, journaux dans CloudWatch. Un conteneur y est naturellement prêt.

Ligne de commande : auto scaling d'un service

\$ Prérequis : le service api existe dans le cluster user7-shop

1. Déclarer le service comme cible, avec ses bornes

```
aws application-autoscaling register-scalable-target --service-namespace ecs \
  --resource-id service/user7-shop/api --scalable-dimension ecs:service:DesiredCount \
  --min-capacity 2 --max-capacity 10
```

2. Suivi de cible : processeur moyen à 60 % ; ajout après 60 s, retrait après 120 s

```
aws application-autoscaling put-scaling-policy --service-namespace ecs \
  --resource-id service/user7-shop/api --scalable-dimension ecs:service:DesiredCount \
  --policy-name cpu-60 --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration '{
    "TargetValue": 60.0,
    "PredefinedMetricSpecification": { "PredefinedMetricType": "ECSServiceAverageCPUUtilization" },
    "ScaleOutCooldown": 60, "ScaleInCooldown": 120 }'
```

3. Action planifiée : au moins 4 tâches en semaine à partir de 7 h (heure de Paris)

```
aws application-autoscaling put-scheduled-action --service-namespace ecs \
  --resource-id service/user7-shop/api --scalable-dimension ecs:service:DesiredCount \
  --scheduled-action-name matin --schedule "cron(0 7 ? * MON-FRI *)" --timezone "Europe/Paris" \
  --scalable-target-action MinCapacity=4,MaxCapacity=10
```

4. Ce que la mise à l'échelle a fait

```
aws application-autoscaling describe-scaling-activities --service-namespace ecs \
  --resource-id service/user7-shop/api --max-items 5 --query 'ScalingActivities[][StartTime,Description,StatusCode]' --output text
```

Ne plus fixer le nombre de tâches à la main. Une fois la mise à l'échelle en place, c'est elle qui décide : un `update-service --desired-count` ou un déploiement d'infrastructure qui réécrit ce nombre annule ses décisions.

Fargate et EC2 : un ou deux niveaux de mise à l'échelle

	Sur Fargate	Sur instances EC2
Niveaux	Un : le nombre de tâches.	Deux : les tâches, et les instances qui les portent.
Si la place manque	Sans objet.	Les tâches attendent qu'une instance soit ajoutée.
Outil	Application Auto Scaling.	Application Auto Scaling, et un fournisseur de capacité qui pilote un Auto Scaling group.
Délai d'ajout	Dizaines de secondes.	Minutes, s'il faut une instance.

Dimensionner tâche et service

- D'abord la **taille de la tâche** : assez de mémoire pour ne jamais être arrêtée, un processeur occupé à 50 ou 60 % en charge normale.
- Ensuite le **nombre** : c'est lui que la mise à l'échelle fait varier.
- Un service qui ne suit pas alors que le processeur est bas : la limite est ailleurs, souvent la base de données ou un service appelé.

Penser à l'aval. Dix tâches ouvrent dix fois plus de connexions à la base qu'une seule. Le maximum du service doit rester compatible avec ce que la base accepte, ou passer par RDS Proxy.

Auto scaling d'un service : erreurs fréquentes

Erreur	Symptôme	Correction
Minimum à une tâche	Coupure à chaque remplacement ou panne de zone.	Deux tâches au moins, sur deux zones.
Maximum trop bas	Saturation sous charge.	Le relever ; alarme quand il est atteint.
Métrique de mémoire sur une application qui ne la rend pas	Le service grossit et ne redescend jamais.	Processeur, ou requêtes par cible.
Tâche sous-dimensionnée en mémoire	Tâches arrêtées sous charge, au pire moment.	Augmenter la mémoire de la tâche.
Nombre de tâches réécrit par l'outil d'infrastructure	Retour brutal à la valeur initiale à chaque déploiement.	Ne pas gérer ce champ dans le code.
Délai de grâce trop court	Nouvelles tâches tuées pendant la montée en charge.	L'allonger.
Base ou service en aval oublié	Plus de tâches, mais pas plus de débit.	Chercher la vraie limite.
Quota de tâches Fargate atteint	Lancements refusés.	Demander une hausse du quota.

Ce qu'il faut retenir

- Un service s'expose par un ALB et un target group de type `ip` ; ECS y inscrit et en retire les tâches.
- `sg-api` n'accepte que `sg-alb`, sur le port du conteneur.
- Le délai de grâce doit couvrir le temps de démarrage de l'application.
- Déployer : nouvelle image, nouvelle révision, mise à jour du service.
- Mise à jour progressive : minimum de tâches saines et maximum encadrent le remplacement.
- Le disjoncteur arrête un déploiement qui échoue et rétablit la révision précédente.
- Application Auto Scaling fait varier le nombre de tâches par suivi de cible.

1. Quel type de target group pour des tâches Fargate ?

2. Qui inscrit les tâches dans le target group ?

3. À quoi sert le délai de grâce ?

4. Que garantit un minimum de 100 % de tâches saines ?

5. Que fait le disjoncteur avec l'option de retour arrière ?

6. Comment revenir en arrière à la main ?

7. Quel service met à l'échelle un service ECS ?

Cliquer sur une question pour afficher la réponse.

Découverte de EKS, partie 1

Découverte d'Amazon EKS

Kubernetes, sans avoir à installer ni à maintenir Kubernetes : AWS exploite le plan de contrôle, vous y déployez vos applications.

- Kubernetes en bref
- Amazon EKS
- Le plan de contrôle
- Les nœuds
- Réseau et exposition
- Créer un cluster

Définition : Kubernetes

Orchestrateur de conteneurs libre, devenu le standard du marché. On lui décrit l'état voulu, dans des fichiers ; il travaille en permanence à le faire correspondre à la réalité.

Définition : pod

Plus petite unité déployable de Kubernetes : un ou plusieurs conteneurs qui partagent le même réseau et la même durée de vie. C'est l'équivalent d'une tâche ECS.

Approche déclarative. On n'ordonne pas « lance trois conteneurs » : on déclare « il doit y en avoir trois », dans un fichier YAML. Kubernetes en relance un s'il en manque, en retire un s'il y en a trop.

Kubernetes	Équivalent ECS	Rôle
Cluster	Cluster	L'ensemble.
Pod	Tâche	Les conteneurs qui tournent.
Deployment	Service	Maintient le nombre voulu de pods, gère les mises à jour.
Service	Target group, Service Connect	Une adresse stable devant des pods.
Ingress	Règles du listener	Routage HTTP depuis l'extérieur.
Namespace	Aucun	Cloisonnement à l'intérieur du cluster.
ServiceAccount	Rôle de tâche	L'identité d'un pod.
Nœud (node)	Instance du cluster	La machine qui porte les pods.

Attention au mot « service » : il ne désigne pas la même chose dans ECS et dans Kubernetes.

Définition : Amazon EKS (Elastic Kubernetes Service)

Service Kubernetes managé. AWS installe, répartit sur plusieurs zones, sauvegarde et met à jour le plan de contrôle ; vous fournissez les nœuds, ou laissez AWS les gérer aussi.

Caractéristique	Détail
Kubernetes conforme	Le Kubernetes d'origine, certifié : les mêmes fichiers, les mêmes outils que partout ailleurs.
Plan de contrôle managé	Haute disponibilité sur trois zones, sans rien régler.
Intégré à AWS	IAM pour l'accès, VPC pour le réseau, load balancers, volumes EBS et EFS.
Coût du plan de contrôle	0,10 \$ par heure et par cluster, soit environ 73 \$ par mois.
Versions	Plusieurs versions de Kubernetes proposées ; une nouvelle environ tous les quatre mois.

Ce qui reste à faire

- Choisir et maintenir les nœuds, sauf en mode automatique.
- Installer et tenir à jour les modules complémentaires.
- Monter de version, régulièrement.
- Maîtriser Kubernetes lui-même : c'est l'essentiel de l'effort.

Managé ne veut pas dire simple. EKS retire l'exploitation du plan de contrôle, pas la complexité de Kubernetes : objets nombreux, réseau, sécurité, écosystème à choisir.

Versions à suivre. Une version de Kubernetes est prise en charge environ 14 mois ; au-delà, un support étendu est facturé six fois plus cher par cluster.

Schéma : plan de contrôle et plan de données

Géré par AWS : plan de contrôle, réparti sur trois zones



Serveur d'API
point d'entrée de kubectl

etcd
l'état du cluster

Ordonnanceur
place les pods

Contrôleurs
rapprochent l'état réel du voulu



les nœuds reçoivent leurs ordres du serveur d'API



Votre VPC : plan de données

Nœud, zone a

pod

pod

pod

Nœud, zone b

pod

pod

Nœud, zone c

pod

pod

pod

Définition : plan de contrôle (control plane)

Cerveau du cluster : le serveur d'API qui reçoit les demandes, la base etcd qui mémorise l'état, l'ordonnanceur et les contrôleurs qui agissent. Dans EKS, AWS l'exploite.

Définition : plan de données (data plane)

Les nœuds, c'est-à-dire les machines où tournent réellement les pods. Ils se trouvent dans votre VPC et dans votre compte.

Les nœuds : quatre façons de les fournir

Option	Principe	Qui gère les machines
EKS Auto Mode	AWS crée, dimensionne, met à jour et remplace les nœuds selon les pods à placer, et gère les modules essentiels.	AWS, entièrement. Le choix le plus simple.
Groupes de nœuds managés	Des Auto Scaling groups d'instances EC2 créés et mis à jour par EKS, à votre demande.	Partagé : vous dimensionnez et déclenchez les mises à jour.
Fargate	Chaque pod tourne dans son propre environnement Fargate, sans nœud visible.	AWS. Des restrictions : pas de DaemonSet, pas de GPU.
Nœuds autogérés	Vos propres instances, inscrites au cluster.	Vous, entièrement.

Définition : EKS Auto Mode

Mode d'EKS dans lequel AWS gère aussi le plan de données : calcul, réseau des pods, stockage et répartition de charge sont configurés et tenus à jour automatiquement.

Pour découvrir EKS, le mode automatique est le plus direct : un cluster utilisable en quelques minutes, sans décider d'un type d'instance. Il ajoute des frais de gestion au prix des instances.

Le réseau des pods

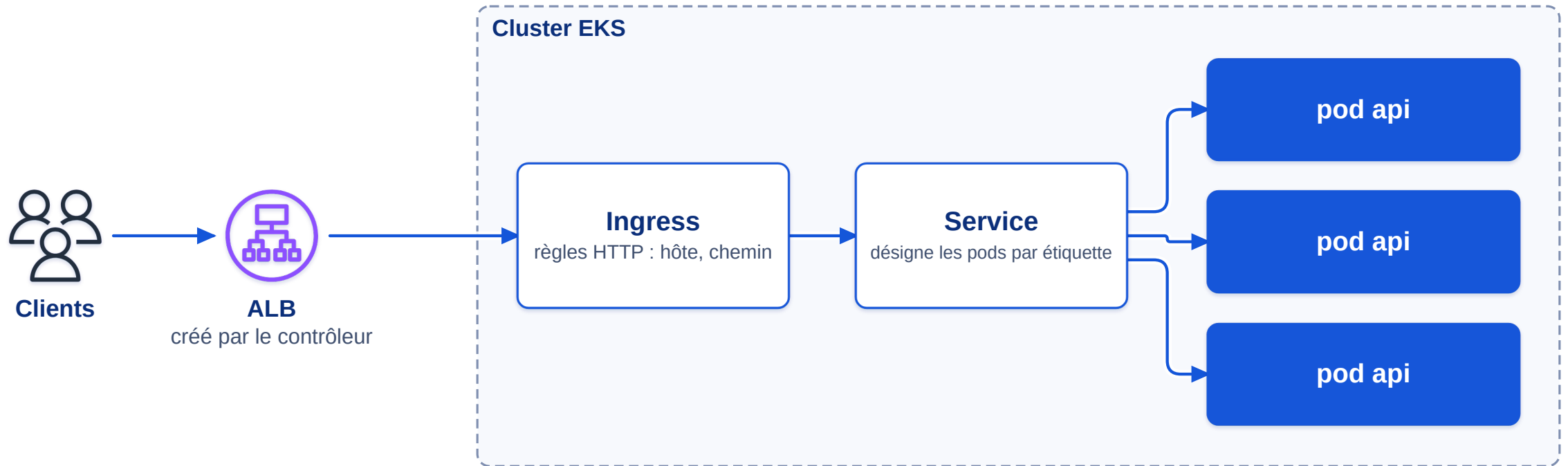
- Avec le module réseau d'AWS, chaque pod reçoit une **adresse IP du VPC**, comme une tâche ECS.
- Les pods se joignent, et joignent les autres ressources du VPC, sans traduction d'adresse.
- Conséquence : un cluster consomme beaucoup d'adresses. Les sous-réseaux doivent être larges.

Plan d'adressage. Des sous-réseaux en /24 pour un cluster actif s'épuisent vite : les pods ne démarrent plus, faute d'adresse. Prévoir des /20 ou plus pour les sous-réseaux des nœuds.

Module (add-on)	Rôle
Amazon VPC CNI	Donne aux pods leurs adresses du VPC.
CoreDNS	Le DNS interne du cluster.
kube-proxy	Le routage vers les Services.
Pilote EBS CSI	Des volumes EBS pour les pods.
AWS Load Balancer Controller	Crée des ALB et des NLB à partir des objets du cluster.
Agent Pod Identity	Des rôles IAM pour les pods.
Observabilité CloudWatch	Métriques et journaux vers CloudWatch.

Définition : module complémentaire (add-on)

Composant installé dans le cluster pour lui apporter une fonction. EKS en propose une liste, qu'il installe et met à jour à la demande.



Un objet Ingress dans le cluster fait naître un load balancer dans AWS.

Objet Kubernetes	Ce qu'il fait naître dans AWS
Service de type LoadBalancer	Un Network Load Balancer.
Ingress	Un Application Load Balancer, avec ses règles.

Le contrôleur fait le lien. Il observe les objets Ingress et Service du cluster et crée, configure et supprime les load balancers correspondants. En mode `ip`, le load balancer envoie directement aux pods, comme un target group de type `ip` vers des tâches.

Un déploiement Kubernetes, pour comparer

yaml api.yaml : l'équivalent d'une task definition et d'un service ECS

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 2
  selector:
    matchLabels: { app: api }
  template:
    metadata:
      labels: { app: api }
    spec:
      serviceAccountName: api-sa
      containers:
        - name: api
          # REGISTRE : adresse complète du registre ECR
          image: REGISTRE/shop/api:1.4.2
          ports:
            - containerPort: 8080
          resources:
            requests: { cpu: 250m, memory: 512Mi }
            limits: { memory: 512Mi }
          readinessProbe:
            httpGet: { path: /health, port: 8080 }
```

Dans ce fichier

Équivalent ECS

replicas: 2

Nombre de tâches voulu.

image , ports

Définition du conteneur.

resources

CPU et mémoire de la tâche.

readinessProbe

Contrôle de santé.

serviceAccountName

Rôle de tâche.

\$ Appliquer et observer

```
kubectl apply -f api.yaml
kubectl get pods
kubectl logs deployment/api
kubectl rollout status deployment/api
```

Mêmes idées, autre vocabulaire. Qui a compris un service ECS reconnaît tout ici. Ce qui change : c'est un standard, lisible par n'importe quel cluster Kubernetes.

\$ Prérequis : AWS CLI connectée, outils eksctl et kubectl installés ; droits d'administration sur EKS, EC2 et IAM ; remplacer user7

1. Créer un cluster en mode automatique, dans un nouveau VPC (15 à 20 minutes)

```
eksctl create cluster --name user7-shop --region eu-west-3 --enable-auto-mode
```

2. Configurer kubectl pour ce cluster (eksctl le fait déjà ; commande utile sur un autre poste)

```
aws eks update-kubeconfig --name user7-shop --region eu-west-3
```

3. Vérifier l'accès : version du cluster, puis nœuds (aucun tant qu'aucun pod n'est déployé, en mode automatique)

```
kubectl version
```

```
kubectl get nodes
```

4. Déployer l'application, puis observer : les nœuds apparaissent pour accueillir les pods

```
kubectl apply -f api.yaml
```

```
kubectl get pods -o wide
```

```
kubectl get nodes
```

5. État du cluster vu d'AWS : version, statut, point d'accès de l'API

```
aws eks describe-cluster --name user7-shop --query 'cluster.[version,status,endpoint]' --output text
```

6. Supprimer le cluster et tout ce qu'il a créé

```
eksctl delete cluster --name user7-shop --region eu-west-3
```

eksctl est l'outil en ligne de commande dédié à EKS : il crée d'un coup le VPC, les rôles, le cluster et les nœuds. La console, Terraform et CloudFormation font la même chose.

Supprimer avant de quitter. Un cluster coûte dès sa création, même vide. Supprimer d'abord les objets Ingress et Service, pour que leurs load balancers disparaissent avec eux.

Ce que coûte un cluster EKS

Poste	Ordre de grandeur
Plan de contrôle	73 \$ par mois et par cluster.
Nœuds	Le prix des instances EC2, ou de Fargate.
Mode automatique	Frais de gestion en plus du prix des instances.
Load balancers	Un par Ingress ou par Service exposé, sauf regroupement.
NAT Gateway, volumes, journaux	Comme pour toute charge.
Support étendu d'une version ancienne	0,60 \$ par heure et par cluster.

Exemple mensuel	Ordre de grandeur
Cluster vide	73 \$
Cluster, deux nœuds t3.medium, un ALB	160 \$
La même application sur ECS et Fargate : deux tâches, un ALB	60 \$

Lecture. Le plan de contrôle est un coût fixe : négligeable pour une plateforme qui héberge des dizaines d'applications, lourd pour une seule petite application.

Un cluster par application multiplie ce coût fixe et le travail de mise à jour. On mutualise : un cluster par environnement, cloisonné par namespaces.

Erreur	Conséquence	Correction
Choisir EKS pour une seule petite application	Coût fixe et complexité sans contrepartie.	ECS sur Fargate.
Sous-réseaux trop petits	Pods bloqués, faute d'adresses.	Sous-réseaux larges pour les nœuds.
Version de Kubernetes jamais mise à jour	Support étendu facturé, puis mise à jour forcée.	Monter de version deux à trois fois par an.
Modules complémentaires oubliés lors d'une montée de version	Incompatibilités, pannes.	Les mettre à jour avec le cluster.
Un load balancer par application	Coûts multipliés.	Regrouper les Ingress sur un même ALB.
Pods sans demandes de ressources	Placement et mise à l'échelle erratiques.	requests sur chaque conteneur.
Cluster de test laissé en place	73 \$ par mois, plus les nœuds.	Le supprimer.
Cluster supprimé avant ses Ingress	Load balancers orphelins, toujours facturés.	Supprimer d'abord les objets du cluster.

Découverte de EKS, partie 2

Accès au cluster et rôles des pods

Deux questions distinctes : qui a le droit de piloter le cluster, et quels droits AWS ont les applications qui y tournent.

- Deux sens, deux mécanismes
- S'authentifier auprès du cluster
- Entrées d'accès et RBAC
- Le problème du rôle du nœud
- EKS Pod Identity et IRSA
- Mise en œuvre

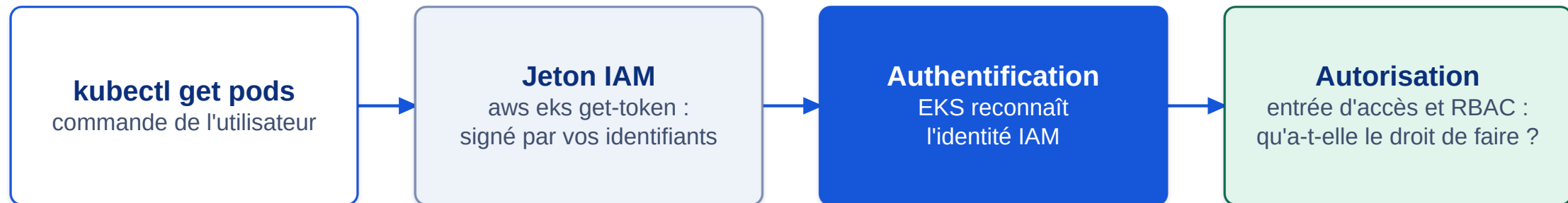
Deux sens, deux mécanismes

	Accéder au cluster	Donner des droits AWS aux pods
Question	Qui peut utiliser <code>kubectl</code> , et pour faire quoi ?	Ce pod peut-il lire ce bucket, cette table ?
Sens	D'une identité IAM vers Kubernetes.	D'un pod Kubernetes vers AWS.
Qui s'authentifie	Un humain, une chaîne d'intégration.	Une application.
Mécanisme	Entrées d'accès du cluster, puis RBAC de Kubernetes.	EKS Pod Identity, ou IRSA.
Équivalent ECS	Les droits IAM sur l'API d'ECS.	Le rôle de tâche.

Deux systèmes de droits coexistent. IAM gouverne AWS ; RBAC gouverne l'intérieur du cluster. EKS fait le pont entre les deux, dans chaque sens.

Les confondre est l'erreur la plus courante. Être administrateur du compte AWS ne donne, par défaut, aucun droit dans un cluster que l'on n'a pas créé. Et être administrateur du cluster ne donne aucun droit sur S3.

Schéma : accéder au cluster avec son identité IAM



Aucun mot de passe propre au cluster : c'est l'identité IAM qui ouvre la porte.

Les droits dans le cluster, eux, restent ceux de Kubernetes.

Définition : kubeconfig

Fichier de configuration de `kubectl` : l'adresse du cluster, son certificat, et la façon d'obtenir un jeton. La commande `aws eks update-kubeconfig` l'écrit pour vous.

Transparent à l'usage. À chaque commande, `kubectl` appelle l'AWS CLI, qui fabrique un jeton de courte durée à partir de vos identifiants courants. Changer de profil AWS, c'est changer d'identité dans le cluster.

Définition : entrée d'accès (access entry)

Déclaration, au niveau du cluster EKS, qu'une identité IAM a le droit d'y entrer. On lui associe une politique d'accès, ou des groupes Kubernetes.

Politique d'accès	Droits dans le cluster
AmazonEKSClusterAdminPolicy	Tout.
AmazonEKSAAdminPolicy	Administration, limitable à des namespaces.
AmazonEKSEditPolicy	Modifier les applications.
AmazonEKSVIEWPolicy	Lecture seule.

Définition : RBAC

Contrôle d'accès de Kubernetes, fondé sur des rôles : il dit quelles actions un utilisateur ou un groupe peut faire sur quels objets, dans quels namespaces.

\$ Donner un accès en lecture à un rôle IAM

```
aws eks create-access-entry --cluster-name user7-shop \  
  --principal-arn arn:aws:iam::123456789012:role/Lecteurs
```

```
aws eks associate-access-policy --cluster-name user7-shop \  
  --principal-arn arn:aws:iam::123456789012:role/Lecteurs \  
  --policy-arn \  
  arn:aws:eks::aws:cluster-access-policy/AmazonEKSVIEWPolicy \  
  --access-scope type=cluster
```

Qui a accès à ce cluster ?

```
aws eks list-access-entries --cluster-name user7-shop
```

Ancien mécanisme. Les clusters plus anciens déclarent leurs accès dans un objet interne, le ConfigMap `aws-auth`. Les entrées d'accès le remplacent : elles se gèrent par l'API d'AWS, sans risque de s'enfermer dehors.

Accès réseau à l'API. Le serveur d'API peut être public, privé, ou les deux. En production, on le rend privé ou on limite les adresses autorisées.

Accès et rôles

Le problème : le rôle du nœud

Un nœud est une instance EC2 : elle porte un rôle d'instance. Sans autre réglage, tous les pods qui tournent dessus peuvent utiliser ce rôle.

Ce que cela implique

- Pour qu'un pod lise un bucket, on ajoute le droit au rôle du nœud.
- Tous les autres pods du nœud, et de tous les nœuds, obtiennent ce droit.
- Au fil des besoins, le rôle du nœud accumule les droits de toutes les applications.

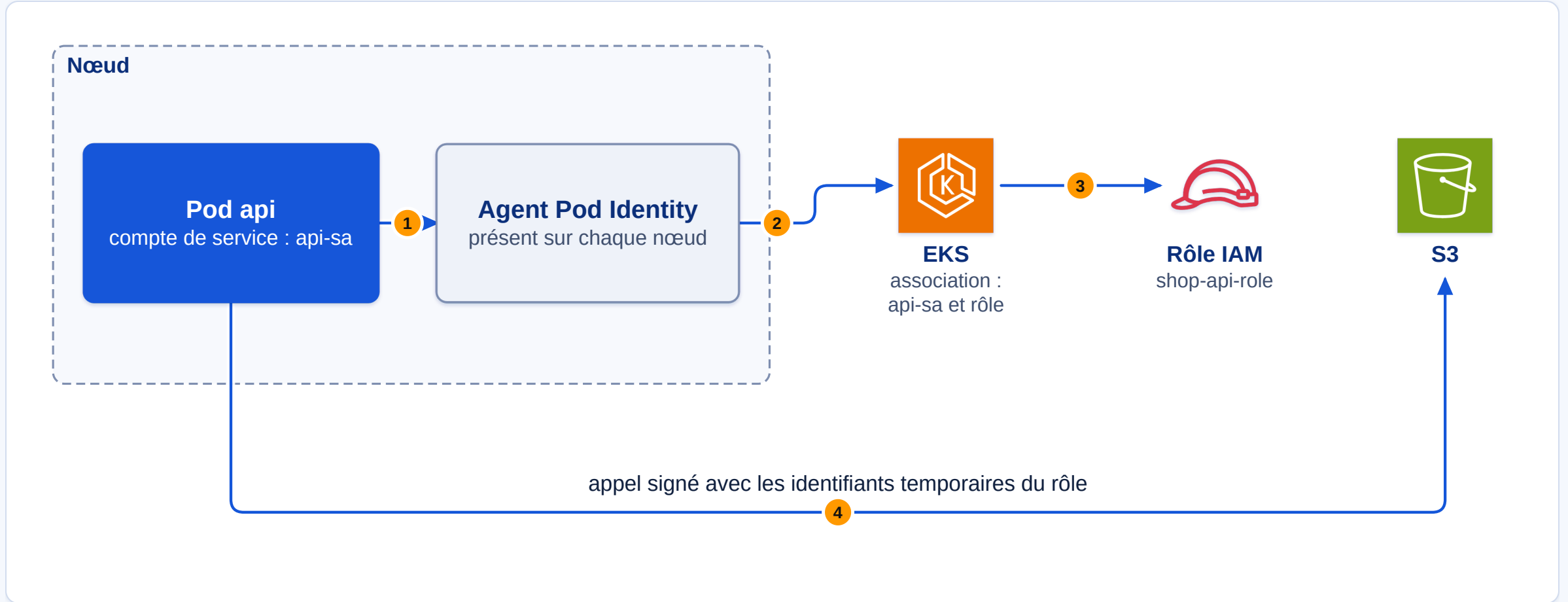
C'est l'inverse du moindre privilège.

	Rôle du nœud	Rôle par pod
Porté par	La machine.	Le compte de service du pod.
Partagé entre	Tous les pods du nœud.	Les pods de cette application seulement.
Doit contenir	Ce dont le nœud a besoin : tirer les images, rejoindre le cluster.	Ce dont l'application a besoin.

Définition : compte de service (ServiceAccount)

Identité Kubernetes attribuée à un pod. C'est à elle que l'on rattache un rôle IAM.

Règle. Le rôle du nœud reste minimal. Chaque application reçoit son propre rôle IAM, par l'un des deux mécanismes de la diapositive suivante. C'est exactement le rôle de tâche d'ECS.



1. Le SDK de l'application, dans le pod, demande des identifiants à l'agent du nœud.

2. L'agent interroge EKS, qui connaît l'association entre le compte de service et un rôle IAM.

3. EKS endosse ce rôle et renvoie des identifiants temporaires.

4. L'application appelle S3 avec les droits de ce rôle, et de lui seul.

	EKS Pod Identity	IRSA (rôles IAM pour comptes de service)
Principe	Une association, déclarée dans EKS, entre un compte de service et un rôle.	Le cluster est un fournisseur d'identité OIDC pour IAM ; le compte de service porte une annotation.
Relation d'approbation du rôle	Le service <code>pods.eks.amazonaws.com</code> : la même pour tous les clusters.	Le fournisseur OIDC du cluster : propre à chaque cluster.
Mise en place	Un module à installer, puis une commande par association.	Un fournisseur OIDC à créer, une relation d'approbation par rôle.
Réutilisation d'un rôle	Simple, d'un cluster à l'autre.	Relation d'approbation à modifier.
Disponibilité	Nœuds EC2 ; inclus en mode automatique.	Partout, y compris Fargate et clusters hors d'AWS.

Choisir. EKS Pod Identity pour un nouveau cluster : plus simple à mettre en place et à administrer. IRSA reste nécessaire pour les pods sur Fargate, et se rencontre dans tous les clusters plus anciens.

Pour l'application, rien ne change. Dans les deux cas, le SDK trouve seul ses identifiants, comme sur une instance ou dans une tâche ECS. Aucune clé dans le code ni dans l'image.

Ligne de commande : un rôle IAM pour un pod

\$ Prérequis : cluster user7-shop ; module eks-pod-identity-agent installé (inclus en mode automatique) ; kubectl configuré

```
# 1. Relation d'approbation : le service Pod Identity peut endosser le rôle
cat > pod-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Principal":{"Service":"pods.eks.amazonaws.com"},
  "Action":["sts:AssumeRole","sts:TagSession"]}]}}
EOF

# 2. Le rôle de l'application, avec ses permissions (ici, lecture de S3 par une politique gérée)
ROLE_ARN=$(aws iam create-role --role-name user7-shop-api-pod \
  --assume-role-policy-document file://pod-trust.json --query Role.Arn --output text)
aws iam attach-role-policy --role-name user7-shop-api-pod \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess

# 3. Le compte de service Kubernetes, puis son association au rôle
kubectl create serviceaccount api-sa --namespace default
aws eks create-pod-identity-association --cluster-name user7-shop \
  --namespace default --service-account api-sa --role-arn $ROLE_ARN

# 4. Vérifier : un pod lancé avec ce compte de service porte bien l'identité du rôle
kubectl run verif --rm -it --restart=Never --image=amazon/aws-cli \
  --overrides='{"spec":{"serviceAccountName":"api-sa"}}' -- sts get-caller-identity
```

À retenir. Le rôle se prépare comme tout rôle de service : une relation d'approbation, des permissions. Seul le principal change : `ec2.amazonaws.com` pour une instance, `ecs-tasks.amazonaws.com` pour une tâche, `pods.eks.amazonaws.com` pour un pod.

Accès et rôles

Accès et rôles : erreurs fréquentes

Message ou symptôme	Cause	Correction
« You must be logged in to the server (Unauthorized) »	L'identité IAM courante n'a aucune entrée d'accès.	Créer l'entrée ; vérifier <code>aws sts get-caller-identity</code> .
« forbidden: User ... cannot list resource »	Entrée présente, droits RBAC insuffisants.	Associer une politique d'accès adaptée.
Délai dépassé sur toute commande <code>kubectl</code>	API privée, poste hors du VPC.	Passer par le VPC, ou autoriser l'adresse.
« AccessDenied » dans le pod, avec le rôle du nœud dans le message	Aucun rôle associé au compte de service.	Créer l'association ; vérifier le compte de service du pod.
Association créée, pod inchangé	Les identifiants sont remis au démarrage.	Recréer le pod.
Droits applicatifs ajoutés au rôle du nœud	Tous les pods en héritent.	Un rôle par application.
Tout le monde administrateur du cluster	Aucune séparation, aucune trace utile.	Politiques d'accès par équipe et par namespace.
Seul le créateur du cluster y a accès	Cluster inaccessible à son départ.	Entrées d'accès pour des rôles, pas pour des personnes.

Découverte de EKS, partie 3

ECS ou EKS

Deux façons de faire tourner les mêmes conteneurs. Le choix dépend moins de la technique que de l'équipe et de son contexte.

- Ce qui est commun
- Ce qui diffère
- Les critères
- Idées reçues
- Les autres options
- S'entraîner

Commun aux deux

- Les mêmes images, tirées du même registre ECR.
- Fargate ou instances EC2 pour la capacité.
- Une adresse du VPC par tâche ou par pod ; les mêmes security groups.
- Les mêmes load balancers, les mêmes certificats.
- Un rôle IAM par application.
- CloudWatch pour les journaux et les métriques.
- Mise à jour progressive, mise à l'échelle automatique.

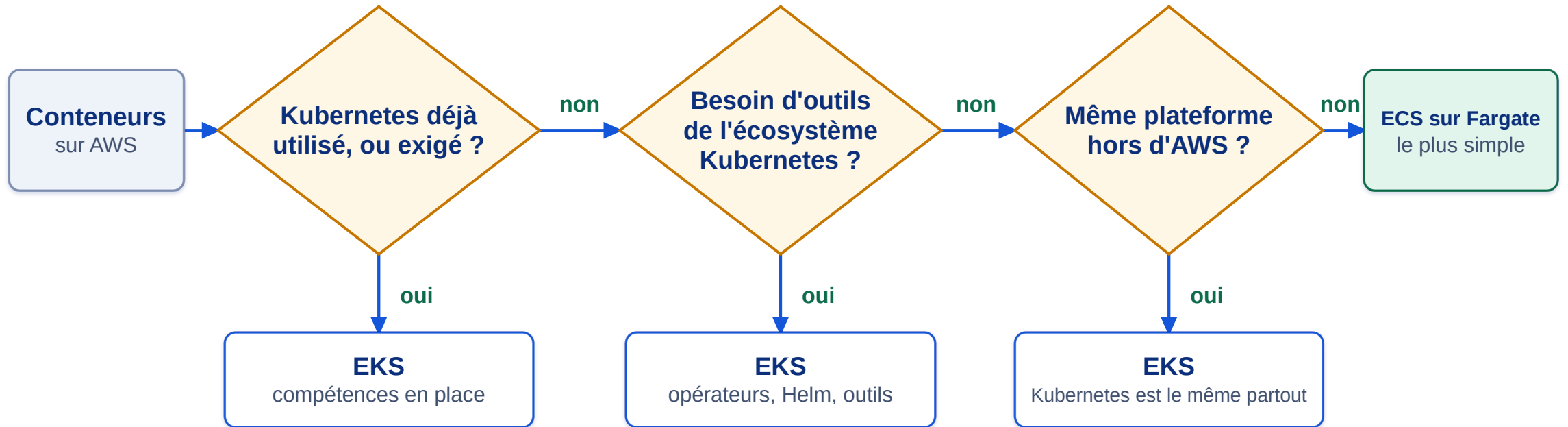
Passer de l'un à l'autre ne remet en cause ni les images, ni le réseau, ni les données.

	ECS	EKS
Origine	Conçu par AWS.	Kubernetes, standard libre.
Notions à apprendre	Quatre.	Des dizaines.
Description	JSON propre à AWS.	YAML standard.
Plan de contrôle	Invisible, gratuit.	Un cluster, 73 \$ par mois.
Mises à jour	Aucune.	Versions et modules à suivre.
Intégration à AWS	Native.	Par des modules.
Écosystème	Celui d'AWS.	Immense : Helm, opérateurs, outils.
Portabilité	AWS uniquement.	Tout fournisseur, et sur site.

Critère	Penche vers ECS	Penche vers EKS
Compétences	Équipe qui découvre les conteneurs, ou qui connaît AWS.	Équipe qui pratique déjà Kubernetes.
Taille	Quelques services, une petite équipe.	Plateforme partagée par de nombreuses équipes.
Effort d'exploitation	Le plus faible possible.	Une équipe plateforme peut s'y consacrer.
Portabilité	Tout est sur AWS, et le restera.	Plusieurs fournisseurs, ou sur site.
Logiciels à déployer	Vos propres applications.	Aussi des produits livrés pour Kubernetes : charts Helm, opérateurs.
Besoins avancés	Rares.	Maillage de services, ordonnancement fin, extensions sur mesure.
Recrutement, marché	Compétence plus rare hors d'AWS.	Compétence très demandée et répandue.
Exigence d'un client ou d'un standard interne	Aucune.	Kubernetes imposé.

Dans le doute : ECS sur Fargate. Il n'y a rien à exploiter, et l'on apprend l'essentiel des conteneurs sans la courbe d'apprentissage de Kubernetes. On passe à EKS le jour où un besoin précis l'exige.

La question décisive. « Avons-nous déjà une raison de vouloir Kubernetes ? » Compétences en place, outils à réutiliser, portabilité exigée : si aucune réponse n'est oui, ECS suffit.



Lecture. Les trois questions cherchent une raison de choisir Kubernetes. S'il n'y en a aucune, la solution la plus simple l'emporte : c'est celle qui laisse le plus de temps pour l'application elle-même.

Idées reçues. « Kubernetes est plus puissant, donc meilleur » : sa puissance se paie en complexité. « ECS enferme chez AWS » : les images et l'application restent portables ; seules les définitions sont à réécrire. « EKS est managé, donc sans effort » : seul le plan de contrôle l'est.

Service	Principe	Quand y penser
ECS sur Fargate	Des services et des tâches, sans machine.	Le cas général de cette formation.
EKS	Kubernetes managé.	Besoin de Kubernetes.
AWS Lambda	Une image de conteneur exécutée à la demande, à chaque événement, facturée à la milliseconde.	Traitements courts, déclenchés par des événements, trafic irrégulier.
AWS Batch	Files de travaux par lots, exécutés dans des conteneurs sur ECS ou EKS.	Calculs lourds, nombreux, sans interface.
Tâche ECS planifiée	Une tâche lancée à heure fixe par un planificateur.	Remplacer un cron.
Instance EC2 avec Docker	Un conteneur lancé à la main sur une machine.	Essai, démonstration. Pas en production.

Une image, plusieurs destinations. La même image peut tourner comme service ECS, comme pod, comme tâche planifiée : l'emballage en conteneur ne préjuge pas de la façon de l'exécuter.

Conteneur ou fonction ? Un service qui tourne en permanence et répond en continu : conteneur. Un traitement bref, déclenché par un événement : fonction. C'est l'objet de la suite du parcours.

S'entraîner : quelle solution pour quel contexte

1. Une équipe de trois développeurs, une API et un site, tout sur AWS.

2. Une entreprise dont les équipes utilisent déjà Kubernetes sur site.

3. Un éditeur dont le logiciel est livré sous forme de chart Helm.

4. Un traitement de dix secondes à chaque fichier déposé dans S3.

5. Un entraînement de modèle sur GPU, pendant plusieurs heures.

6. Une application qui doit tourner à l'identique chez deux fournisseurs de cloud.

7. Un export comptable chaque nuit à 2 heures.

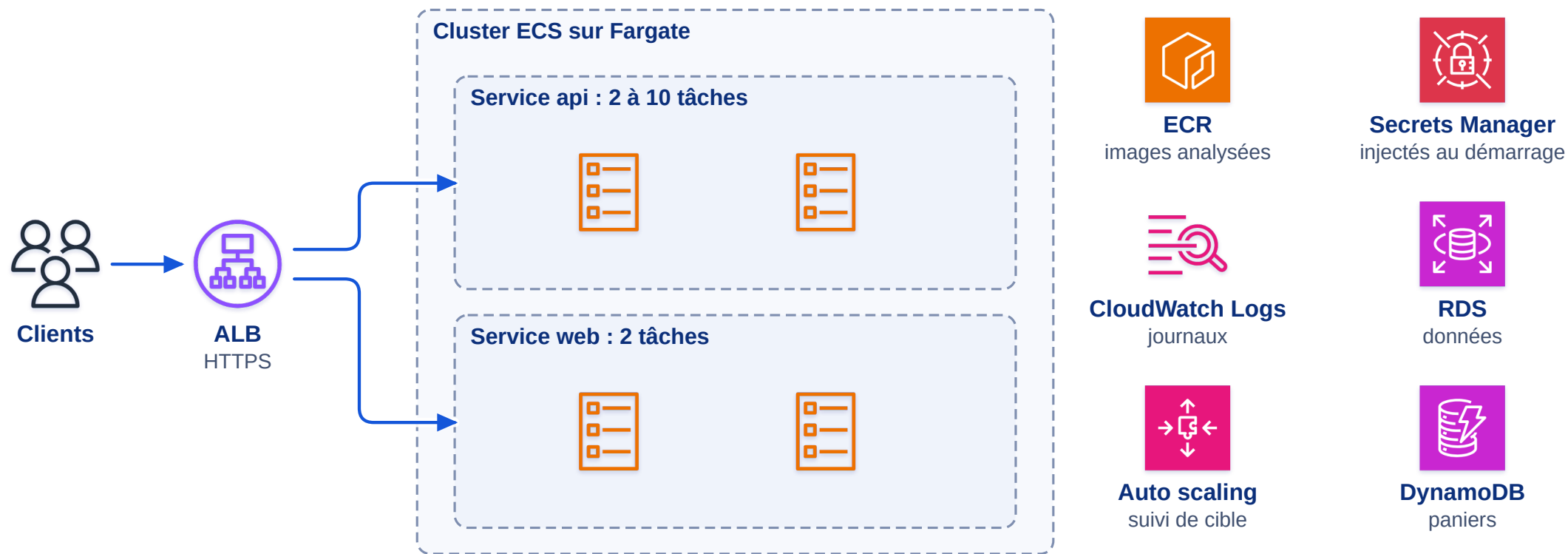
8. Cinquante équipes, deux cents services, une équipe plateforme dédiée.

9. Une jeune entreprise qui veut livrer vite, sans personne pour l'exploitation.

10. Un conteneur qui exige le mode privilégié et l'accès au matériel.

Cliquer sur une question pour afficher la réponse.

Erreur	Conséquence	Correction
Choisir Kubernetes pour le CV, ou par mode	Des mois passés sur la plateforme, pas sur le produit.	Partir du besoin.
Sous-estimer l'exploitation d'EKS	Cluster jamais mis à jour, modules obsolètes.	Prévoir le temps, ou rester sur ECS.
Rejeter ECS par crainte de l'enfermement	Complexité subie pour une portabilité jamais utilisée.	Évaluer le coût réel d'une migration : les images restent.
Un cluster EKS par application	Coûts fixes et mises à jour multipliés.	Mutualiser, cloisonner par namespaces.
Comparer sur le seul prix du plan de contrôle	Oubli du coût humain, de loin le plus lourd.	Compter le temps d'équipe.
Mettre en conteneur ce qui serait mieux en fonction	Service permanent pour dix appels par jour.	Lambda.
Migrer vers les conteneurs sans rendre l'application sans état	Sessions et fichiers perdus à chaque déploiement.	Sortir l'état d'abord.



Images

Construites par la chaîne d'intégration, poussées dans ECR sous un tag unique, analysées à chaque envoi, purgées par une règle de cycle de vie.

Exécution

Deux services ECS sur Fargate, dans les sous-réseaux privés, derrière un ALB ; disjoncteur de déploiement et suivi de cible sur le processeur.

Autour

Un rôle de tâche par service, les secrets injectés depuis Secrets Manager, les journaux dans CloudWatch Logs avec une rétention.

Ce qu'il faut retenir

- Kubernetes est le standard de l'orchestration ; EKS en exploite le plan de contrôle.
- Les nœuds sont dans votre VPC : mode automatique, groupes managés, Fargate, ou autogérés.
- Chaque pod a une adresse du VPC ; un Ingress fait naître un ALB.
- On accède au cluster avec son identité IAM, par une entrée d'accès ; RBAC fixe les droits à l'intérieur.
- Un pod reçoit ses droits AWS par EKS Pod Identity ou IRSA, jamais par le rôle du nœud.
- ECS : simple, intégré, sans exploitation. EKS : standard, portable, plus exigeant.
- Sans raison de vouloir Kubernetes : ECS sur Fargate.

1. Que gère AWS dans un cluster EKS ?

2. Quel est l'équivalent Kubernetes d'une tâche ECS ?

3. Que coûte un cluster EKS vide ?

4. Comment kubectl s'authentifie-t-il auprès d'EKS ?

5. Pourquoi ne pas donner les droits applicatifs au rôle du nœud ?

6. Quel principal figure dans la relation d'approbation d'un rôle Pod Identity ?

7. Quel choix par défaut entre ECS et EKS ?

Cliquer sur une question pour afficher la réponse.

#	Pratique	Chapitre
1	Images petites, à version précise, sans secret, exécutées sans privilège.	Images et registre
2	Un tag unique par construction, dans un dépôt aux tags immuables.	Images et registre
3	Scan à l'envoi et règle de cycle de vie sur chaque dépôt.	Images et registre
4	Architecture fixée par <code>--platform</code> ; images multi-architecture.	Images et registre
5	Fargate par défaut ; tâches dans les sous-réseaux privés, sans adresse publique.	ECS sur Fargate
6	Un rôle de tâche par application ; secrets par référence, jamais en clair.	ECS sur Fargate
7	Journaux sur la sortie standard, vers CloudWatch Logs, avec une rétention.	ECS sur Fargate
8	Au moins deux tâches sur deux zones, derrière un load balancer, avec un contrôle de santé dédié.	Exploiter un service ECS
9	Disjoncteur de déploiement et retour arrière activés sur tous les services.	Exploiter un service ECS
10	Suivi de cible sur chaque service ; maximum compatible avec la base.	Exploiter un service ECS
11	EKS seulement pour une raison précise ; un rôle par pod ; versions tenues à jour.	Découverte de EKS

Fil conducteur. Une image construite une fois, identique partout ; des conteneurs jetables, remplacés et jamais modifiés ; et une plateforme qui se charge de les maintenir en nombre, de les relier et de les déployer.

Terme	Définition
Conteneur	Processus isolé, avec son système de fichiers et ses dépendances.
Image	Modèle en lecture seule d'un conteneur.
Couche	Fichiers ajoutés par une instruction du Dockerfile.
Dockerfile	Recette de construction d'une image.
Registre	Service qui stocke et distribue des images.
Dépôt	Emplacement d'un registre pour les versions d'une image.
Tag	Étiquette lisible d'une version d'image.
Empreinte (digest)	Identifiant calculé sur le contenu d'une image.
Amazon ECR	Registre d'images privé d'AWS.
Scan d'image	Recherche de vulnérabilités connues dans une image.
Tags immuables	Interdiction de réutiliser un tag.
Règle de cycle de vie	Suppression automatique d'anciennes images.
Architecture	Type de processeur visé : x86_64 ou ARM64.
Image multi-architecture	Tag regroupant une variante par architecture.

Terme	Définition
Orchestrateur	Système qui lance, surveille et relie des conteneurs.
Cluster ECS	Regroupement de services et de tâches.
Task definition	Description versionnée d'une tâche.
Tâche	Exécution d'une task definition.
Service	Maintient un nombre voulu de tâches.
Fargate	Exécution de conteneurs sans machine à gérer.
Mode awsvpc	Une interface réseau par tâche.
Rôle d'exécution	Rôle utilisé par ECS pour démarrer la tâche.
Rôle de tâche	Rôle utilisé par l'application.
Pilote awslogs	Envoi de la sortie standard vers CloudWatch Logs.
ECS Exec	Terminal dans un conteneur en marche.
Mise à jour progressive	Remplacement des tâches par étapes.
Disjoncteur de déploiement	Arrêt et retour arrière d'un déploiement en échec.
Application Auto Scaling	Variation automatique du nombre de tâches.
Fournisseur de capacité	Origine de la capacité : Fargate, Fargate Spot, instances.

Synthèse

Glossaire (3/3) : EKS

Terme	Définition
Kubernetes	Orchestrateur de conteneurs libre, standard du marché.
Amazon EKS	Kubernetes managé par AWS.
Plan de contrôle	Serveur d'API, etcd, ordonnanceur, contrôleurs.
Plan de données	Les nœuds où tournent les pods.
Nœud	Machine du cluster.
Pod	Un ou plusieurs conteneurs lancés ensemble.
Deployment	Maintient un nombre voulu de pods.
Service (Kubernetes)	Adresse stable devant des pods.
Ingress	Règles de routage HTTP vers les Services.
Namespace	Cloisonnement à l'intérieur d'un cluster.
EKS Auto Mode	Nœuds et modules essentiels gérés par AWS.
Entrée d'accès	Droit d'une identité IAM d'entrer dans un cluster.
RBAC	Contrôle d'accès par rôles de Kubernetes.
Compte de service	Identité Kubernetes d'un pod.
EKS Pod Identity, IRSA	Rattachement d'un rôle IAM à un compte de service.

Sujet	Documentation officielle
Amazon ECR	Amazon ECR User Guide
Amazon ECS	Amazon ECS Developer Guide
Bonnes pratiques ECS	Amazon ECS Best Practices Guide
Amazon EKS	Amazon EKS User Guide
Bonnes pratiques EKS	Amazon EKS Best Practices Guide
Kubernetes	kubernetes.io/fr/docs
Tarifs	aws.amazon.com/fargate/pricing

Icônes. Les icônes des schémas proviennent du jeu officiel « AWS Architecture Icons », publié par AWS pour la réalisation de schémas d'architecture.

Ce qui est acquis

- Construire une image, la pousser dans ECR, la tenir saine.
- Décrire et lancer un service sur ECS et Fargate.
- L'exposer, le déployer sans coupure, le mettre à l'échelle.
- Situer EKS, et choisir entre les deux.

Suite du parcours : sans serveur et intégration

- Les fonctions et les API.
- Les files de messages et les notifications.
- Les événements.

Après les conteneurs qui tournent en permanence, le code qui ne s'exécute que lorsqu'un événement l'appelle.

Avant de quitter. Mettre les services à zéro tâche puis les supprimer, supprimer les clusters ECS et EKS, les load balancers, les dépôts ECR, les groupes de journaux et les rôles créés pour l'exercice.